CARAGA STATE UNIVERSITY
College of Computing and Information Sciences
Mechatronics Building, CSU-Main Campus, Ampayon, Butuan City, Philippines
Competence Service Uprightness
Your Best Option to Success!
Phone (+63 85) 341-3249
(+63 85) 342-3047
(+63 85) 341-2296
Phone: 341-2786 loc. 123
Email Address: ccis@carsu.edu.ph
URL: https://ccis.carsu.edu.ph/

ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 1) LIGHTS

COURSE/SUBJECT: *APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES*
COURSE NO.: *ITE-18*
INSTRUCTOR/PROF: *JAMES EARL D. CUBILLAS, M.Sc., ECT*

Introduction

As we saw in the previous lesson, adding lights is as simple as adding meshes. You instantiate a light using the proper class, and you add it to the scene.

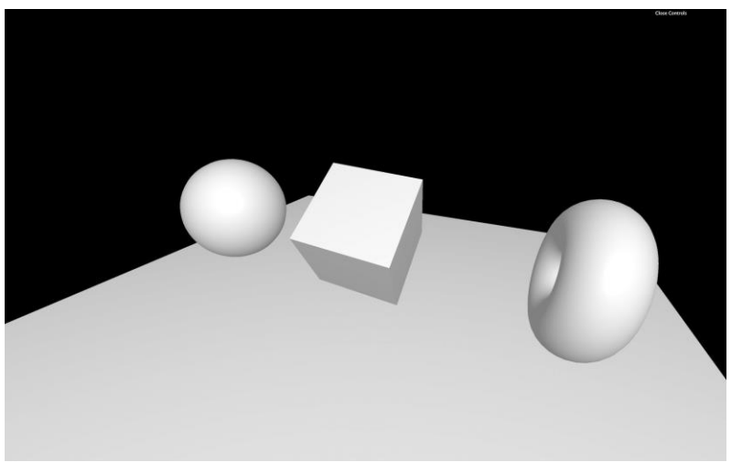
There are multiple types of lights, and we already discovered the AmbientLight and the PointLight.

In this lesson, we will see all the different classes in detail and how to use them.

Setup

A scene is already set up in the starter (complete with a sphere, a cube, a torus, and a plane as the floor), but feel free to try this yourself if you want to practice.

Because we are going to use lights, we must use a material that reacts to lights. We could have used MeshLambertMaterial, MeshPhongMaterial or MeshToonMaterial, but instead we will use the MeshStandardMaterial because it's the most realistic one as we saw in the previous lesson. We also reduced the **roughness** of the material to **0.4** to see the reflections of the lights.



Once the starter is working remove the AmbientLight and the PointLight to start from scratch. You should get a black render with nothing visible in it.

AmbientLight

The AmbientLight applies omnidirectional lighting on all geometries of the scene. The first parameter is the **color** and the second parameter is the **intensity**. As for the materials, you can set the properties directly when instantiating or you can change them after:

```
const ambientLight = new THREE.AmbientLight(0xffffff, 0.5)
scene.add(ambientLight)

// Equals
const ambientLight = new THREE.AmbientLight()
ambientLight.color = new THREE.Color(0xffffff)
ambientLight.intensity = 0.5
scene.add(ambientLight)
```



And like we did for the materials, you can add the properties to the Debug UI. We won't do that in the rest of the lesson but feel free to add tweaks if you want to ease the testing:

```
gui.add(ambientLight, 'intensity').min(0).max(1).step(0.001)
```

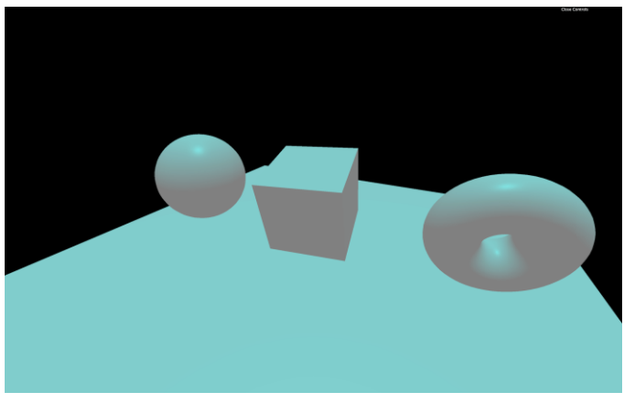
If all you have is an AmbientLight you'll have the same effect as for a MeshBasicMaterial because all faces of the geometries will be lit equally.

In real life, when you light up an object, the sides of the objects at the opposite of the light won't be totally black because light bounces on the walls and other objects. Light bouncing is not supported in Three.js for performance reasons, but you can use a dim AmbientLight to fake this light bounce.

DirectionalLight

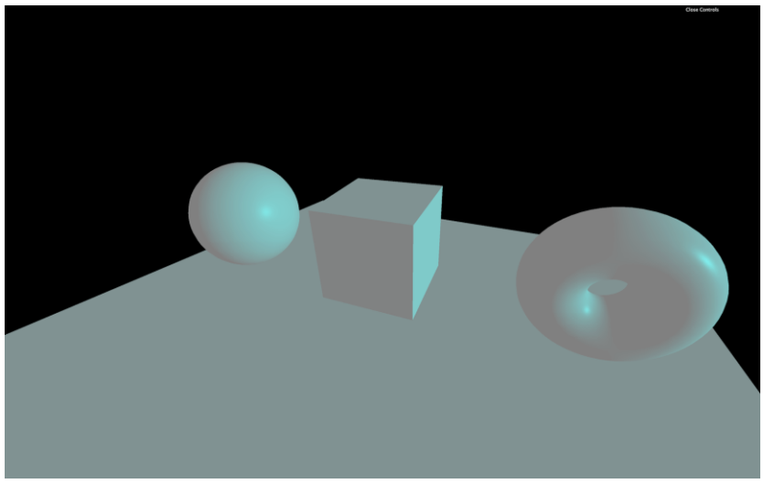
The DirectionalLight will have a sun-like effect as if the sun rays were traveling in parallel. The first parameter is the **color** and the second parameter is the **intensity**:

```
const directionalLight = new THREE.DirectionalLight(0x00fffc, 0.3)
scene.add(directionalLight)
```



By default, the light will seems to come from above. To change that, you must move the whole light by using the **position** property like if it were a normal object.

```
directionalLight.position.set(1, 0.25, 0)
```



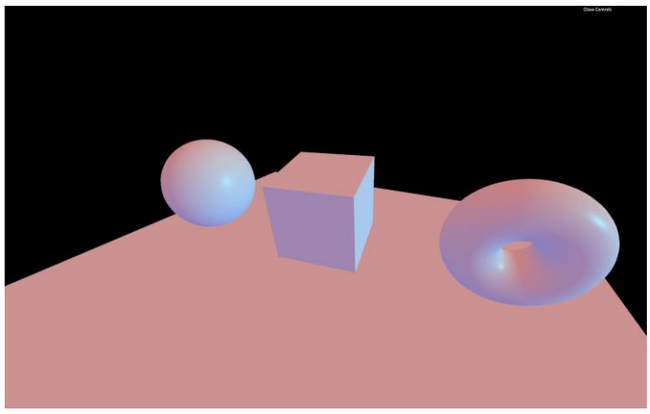
The distance of the light doesn't matter for now. The rays come from an infinite space and travel in parallel to the infinite opposite.

HemisphereLight

The HemisphereLight is similar to the AmbientLight but with a different color from the sky than the color coming from the ground. Faces facing the sky will be lit by one color while another color will lit faces facing the ground.

The first parameter is the **color** corresponding to the sky color, the second parameter is the **groundColor** and the third parameter is the **intensity**:

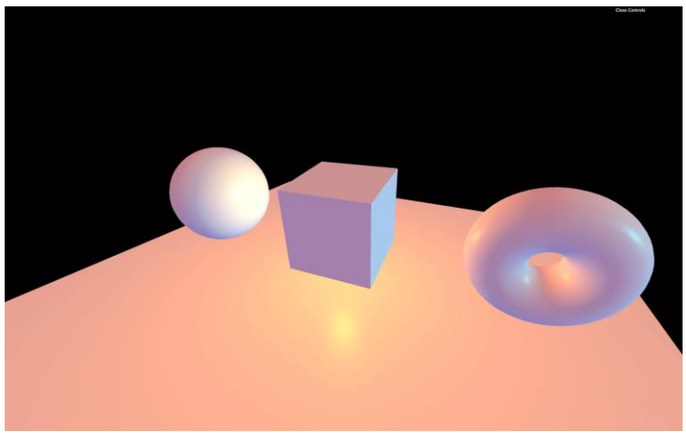
```
const hemisphereLight = new THREE.HemisphereLight(0xff0000, 0x0000ff, 0.3)
scene.add(hemisphereLight)
```



PointLight

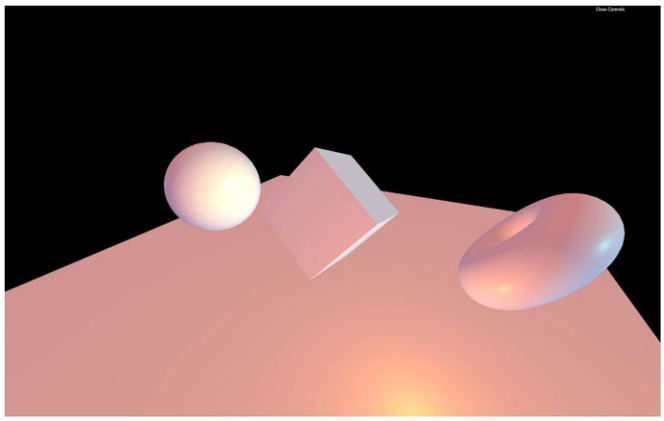
The PointLight is almost like a lighter. The light source is infinitely small, and the light spreads uniformly in every direction. The first parameter is the **color** and the second parameter is the **intensity**:

```
const pointLight = new THREE.PointLight(0xff9000, 0.5)
scene.add(pointLight)
```



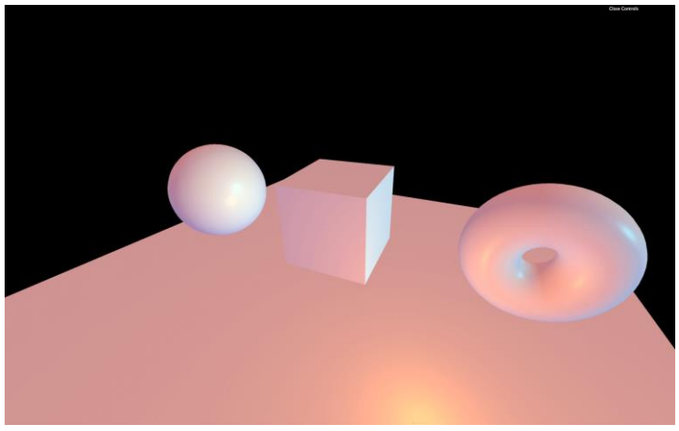
We can move it like any object:

```
pointLight.position.set(1, - 0.5, 1)
```



By default, the light intensity doesn't fade. But you can control that fade distance and how fast it is fading using the **distance** and **decay** properties. You can set those in the parameters of the class as the third and fourth parameters, or in the properties of the instance:

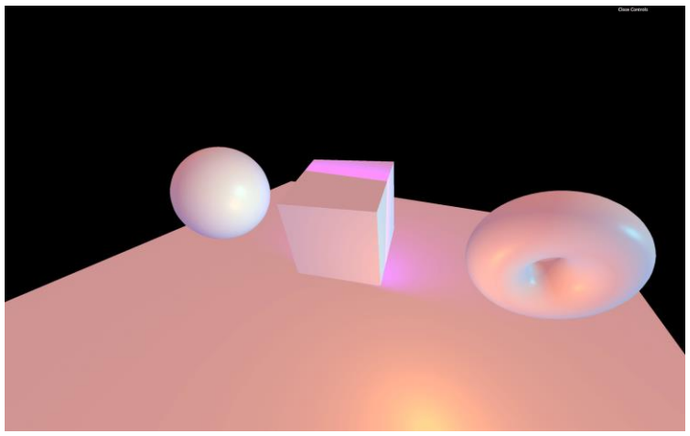
```
const pointLight = new THREE.PointLight(0xff9000, 0.5, 10, 2)
```



RectAreaLight

The RectAreaLight works like the big rectangle lights you can see on the photoshoot set. It's a mix between a directional light and a diffuse light. The first parameter is the **color**, the second parameter is the **intensity**, the third parameter is **width** of the rectangle, and the fourth parameter is its **height**:

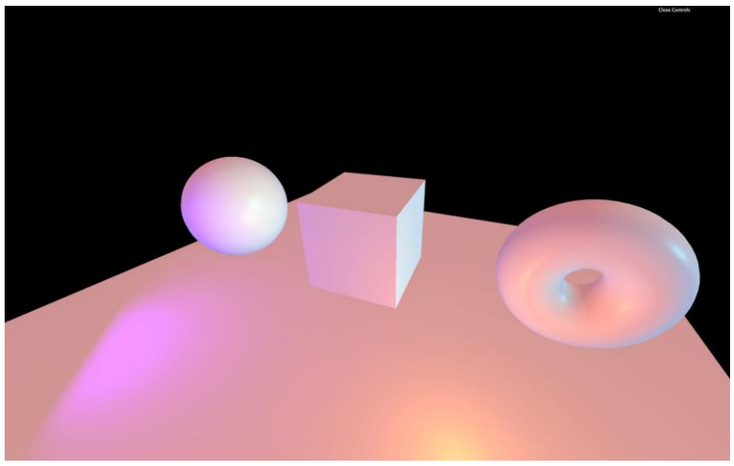
```
const rectAreaLight = new THREE.RectAreaLight(0x4e00ff, 2, 1, 1)
scene.add(rectAreaLight)
```



The RectAreaLight only works with MeshStandardMaterial and MeshPhysicalMaterial.

You can then move the light and rotate it. To ease the rotation, you can use the **lookAt(...)** method that we saw in a previous lesson:

```
rectAreaLight.position.set(- 1.5, 0, 1.5)
rectAreaLight.lookAt(new THREE.Vector3())
```



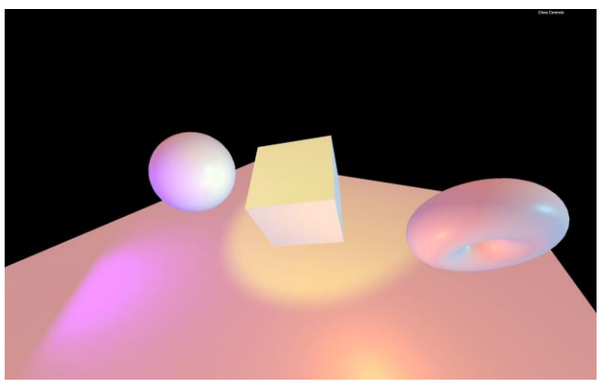
A Vector3 without any parameter will have its **x**, **y**, and **z** to **0** (the center of the scene).

SpotLight

The SpotLight works like a flashlight. It's a cone of light starting at a point and oriented in a direction. Here the list of its parameters:

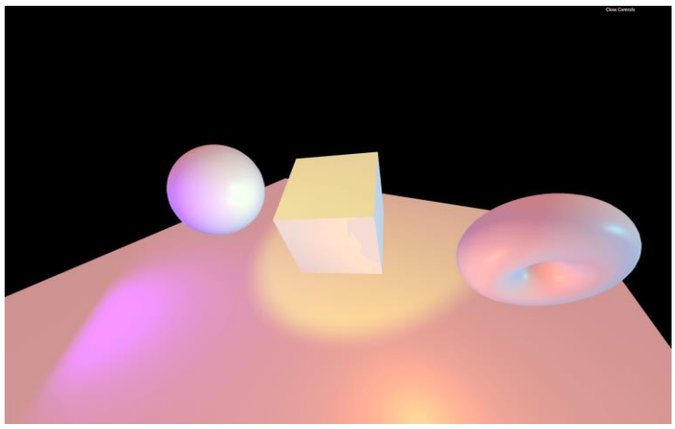
- **color**: the color
- **intensity**: the strength
- **distance**: the distance at which the intensity drops to **0**
- **angle**: how large is the beam
- **penumbra**: how diffused is the contour of the beam
- **decay**: how fast the light dims

```
const spotLight = new THREE.SpotLight(0x78ff00, 0.5, 10, Math.PI * 0.1,
0.25, 1)
spotLight.position.set(0, 2, 3)
scene.add(spotLight)
```



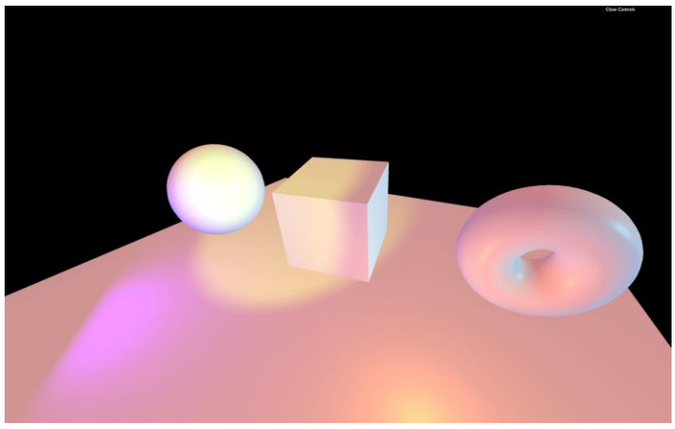
Rotating our SpotLight is a little harder. The instance has a property named **target**, which is an Object3D. The SpotLight is always looking at that **target** object. But if you try to change its position, the SpotLight won't budge:

```
spotLight.target.position.x = - 0.75
```



That is due to our **target** not being in the scene. Simply add the **target** to the scene, and it should work:

```
scene.add(spotLight.target)
```



Performance

Lights are great and can be realistic if well used. The problem is that lights can cost a lot when it comes to performance. The GPU will have to do many calculations like the distance from the face to the light, how much that face is facing the light, if the face is in the spot light cone, etc.

Try to add as few lights as possible and try to use the lights that cost less.

Minimal cost:

- AmbientLight
- HemisphereLight

Moderate cost:

- DirectionalLight
- PointLight

High cost:

- SpotLight
- RectAreaLight

Helpers

Positioning and orienting the lights is hard. To assist us, we can use helpers. Only the following helpers are supported:

- HemisphereLightHelper
- DirectionalLightHelper
- PointLightHelper
- RectAreaLightHelper
- SpotLightHelper

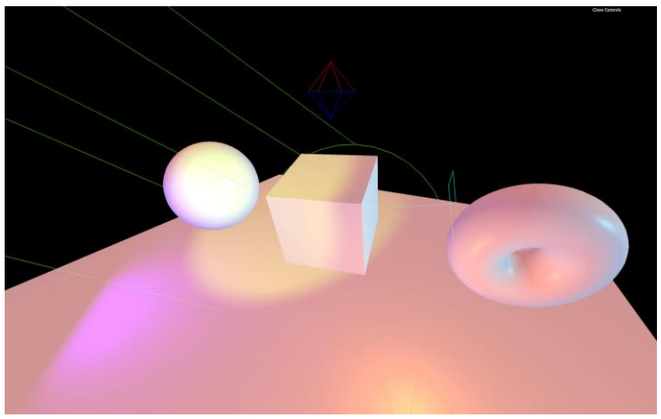
To use them, simply instantiate those classes. Use the corresponding light as a parameter, and add them to the scene. The second parameter enables you to change the helper's **size**:

```
const hemisphereLightHelper = new
THREE.HemisphereLightHelper(hemisphereLight, 0.2)
scene.add(hemisphereLightHelper)

const directionalLightHelper = new
THREE.DirectionalLightHelper(directionalLight, 0.2)
scene.add(directionalLightHelper)

const pointLightHelper = new THREE.PointLightHelper(pointLight, 0.2)
scene.add(pointLightHelper)

const spotLightHelper = new THREE.SpotLightHelper(spotLight)
scene.add(spotLightHelper)
```

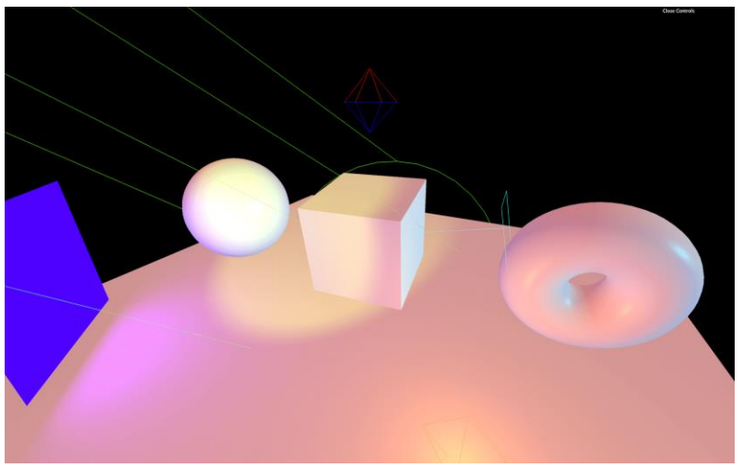


The RectAreaLightHelper is a little harder to use. Right now, the class isn't part of the **THREE** core variables. You must import it from the **examples** dependencies as we did with OrbitControls:

```
import { RectAreaLightHelper } from
'three/examples/jsm/helpers/RectAreaLightHelper.js'
```

Then you can use it:

```
const rectAreaLightHelper = new RectAreaLightHelper(rectAreaLight)
scene.add(rectAreaLightHelper)
```


Close Controls

CARAGA STATE UNIVERSITY
College of Computing and Information Sciences
Mechatronics Building, CSU-Main Campus, Ampayon, Butuan City, Philippines
Competence Service Uprightness
Your Best Option to Success!
Phone (+63 85) 341-3249
(+63 85) 342-3047
(+63 85) 341-2296
Phone: 341-2786 loc. 123
Email Address: ccis@carsu.edu.ph
URL: https://ccis.carsu.edu.ph/

ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 2) SHADOWS

COURSE/SUBJECT: *APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES*
COURSE NO.: *ITE-18*
INSTRUCTOR/PROF: *JAMES EARL D. CUBILLAS, M.Sc., ECT*

Introduction

Now that we have lights, we want shadows. The back of the objects is in the dark, which is called the **core shadow**. We are missing the **drop shadow**, where objects create shadows on the other objects.

Shadows have always been a challenge for real-time 3D rendering, and developers must find tricks to display realistic shadows at a reasonable frame rate.

There are many ways of implementing them, and Three.js has a built-in solution. Be aware that this solution is convenient, but it's far from perfect.

How it works

We won't detail how shadows are working internally, but we will try to understand the basics.

When you do one render, Three.js will first do a render for each light supposed to cast shadows. Those renders will simulate what the light sees as if it was a camera. During these lights renders, MeshDepthMaterial replaces all meshes materials.

The results are stored as textures and named shadow maps.

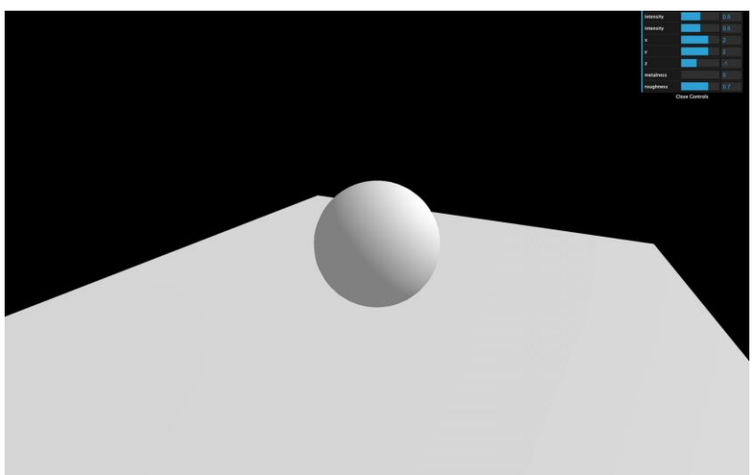
You won't see those shadow maps directly, but they are used on every material supposed to receive shadows and projected on the geometry.

Here's an excellent example of what the directional light and the spotlight see: https://threejs.org/examples/webgl_shadowmap_viewer.html

Setup

Our starter is composed of one simple sphere on a plane with one directional light and one ambient light.

You can control these lights and the material metalness and roughness in Dat.GUI.



How to activate shadows

First, we need to activate the shadow maps on the **renderer**:

```
renderer.shadowMap.enabled = true
```

Then, we need to go through each object of the scene and decide if the object can cast a shadow with the **castShadow** property, and if the object can receive shadow with the **receiveShadow** property.

Try to activate these on as few objects as possible:

```
sphere.castShadow = true

// ...
plane.receiveShadow = true
```

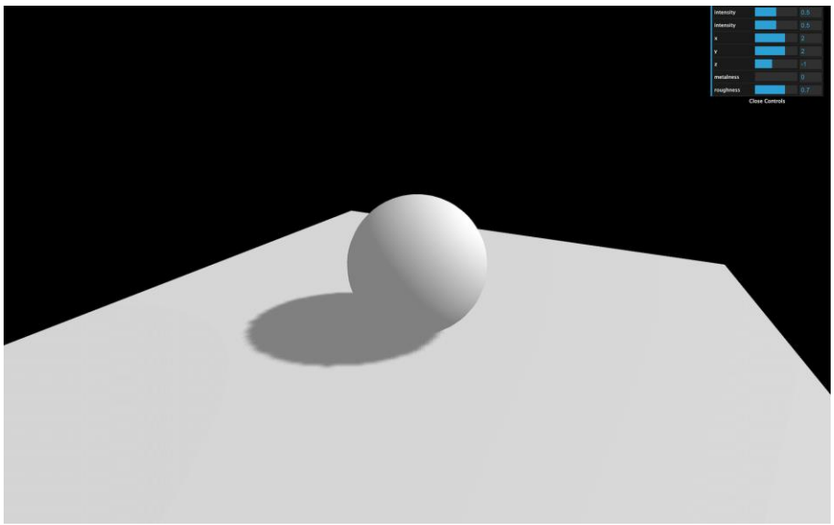
Finally, activate the shadows on the light with the **castShadow** property.

Only the following types of lights support shadows:

- PointLight
- DirectionalLight
- SpotLight

And again, try to activate shadows on as few lights as possible:

```
directionalLight.castShadow = true
```



You should get a shadow of the sphere on the plane.

Sadly, that shadow looks terrible. Let's try to improve it.

Shadow map optimisations

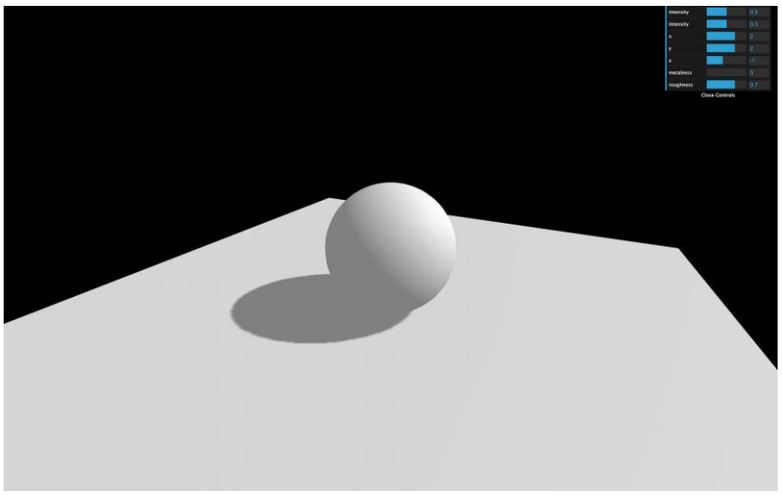
Render size

As we said at the start of the lesson, Three.js is doing renders called shadow maps for each light. You can access this shadow map (and many other things) using the **shadow** property on the light:

```
console.log(directionalLight.shadow)
```

As for our render, we need to specify a size. By default, the shadow map size is only **512x512** for performance reasons. We can improve it but keep in mind that you need a power of 2 value for the mipmapping:

```
directionalLight.shadow.mapSize.width = 1024
directionalLight.shadow.mapSize.height = 1024
```



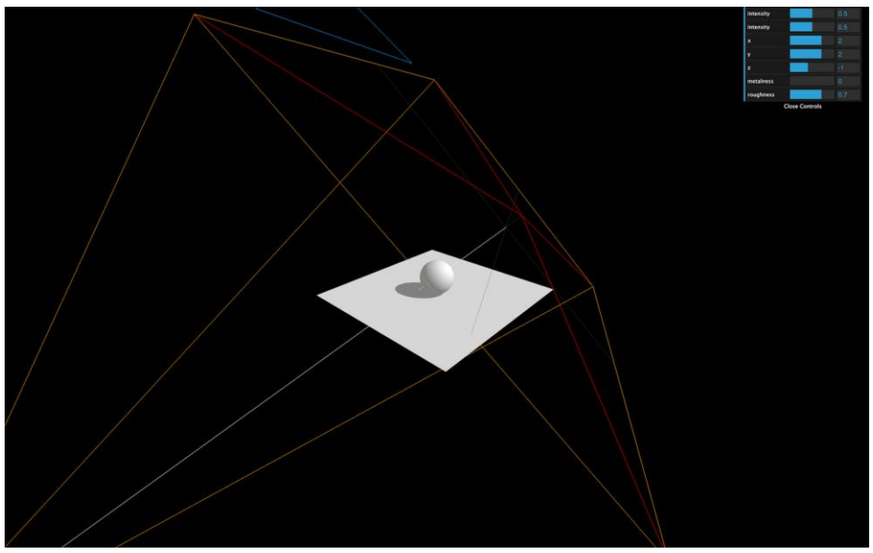
The shadow should already look better.

Near and far

Three.js is using cameras to do the shadow maps renders. Those cameras have the same properties as the cameras we already used. This means that we must define a **near** and a **far**. It won't really improve the shadow's quality, but it might fix bugs where you can't see the shadow or where the shadow appears suddenly cropped.

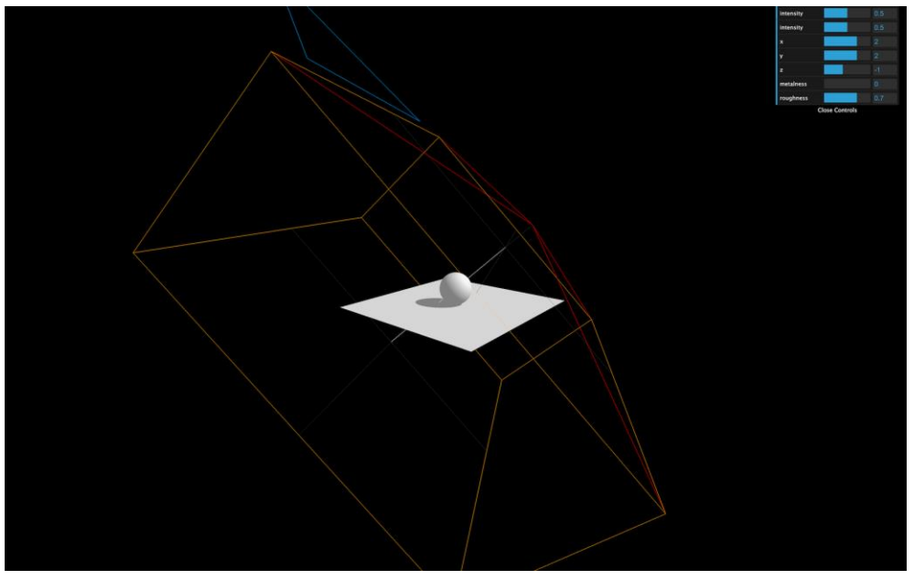
To help us debug the camera and preview the **near** and **far**, we can use a CameraHelper with the camera used for the shadow map located in the **directionalLight.shadow.camera** property:

```
const directionalLightCameraHelper = new
THREE.CameraHelper(directionalLight.shadow.camera)
scene.add(directionalLightCameraHelper)
```



Now you can visually see the **near** and **far** of the camera. Try to find a value that fits the scene:

```
directionalLight.shadow.camera.near = 1
directionalLight.shadow.camera.far = 6
```

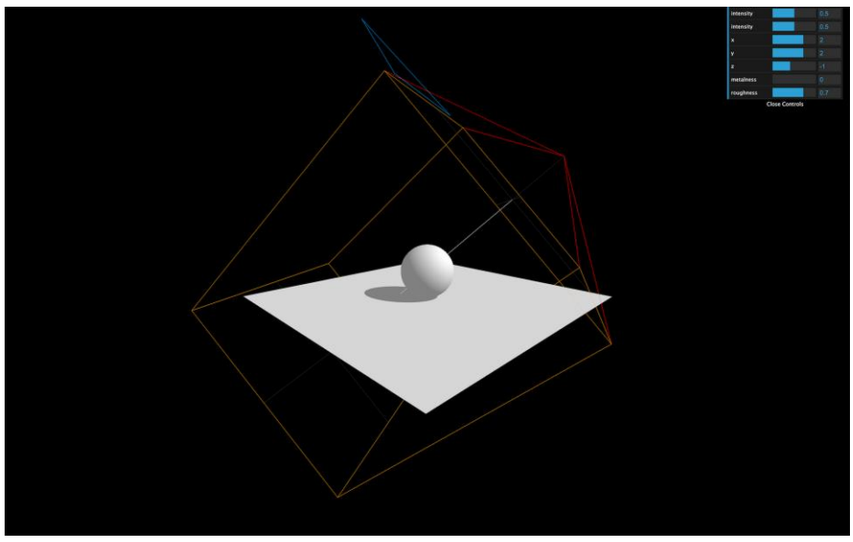


Amplitude

We can see that the camera's amplitude is too large with the camera helper we just added.

Because we are using a DirectionalLight, Three.js is using an OrthographicCamera. If you remember from the Cameras lesson, we can control how far on each side the camera can see with the **top**, **right**, **bottom**, and **left** properties. Let's reduce those properties:

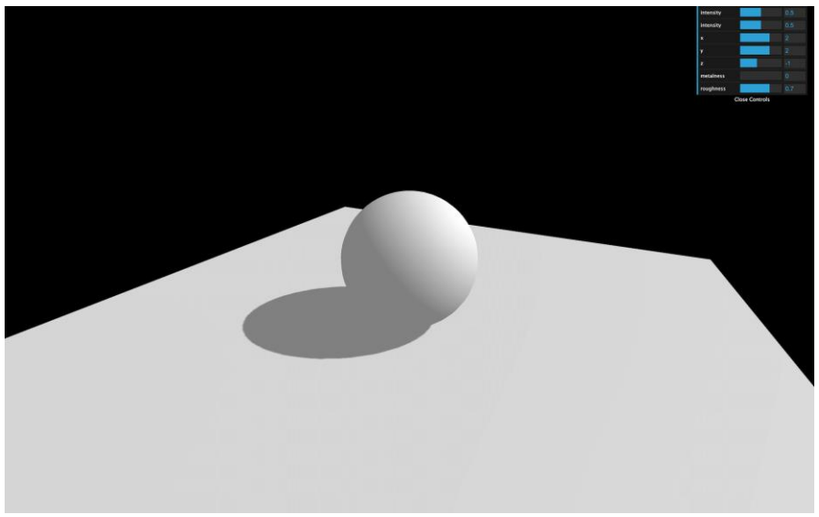
```
directionalLight.shadow.camera.top = 2
directionalLight.shadow.camera.right = 2
directionalLight.shadow.camera.bottom = - 2
directionalLight.shadow.camera.left = - 2
```



The smaller the values, the more precise the shadow will be. But if it's too small, the shadows will just be cropped.

You can hide the camera helper once you're done:

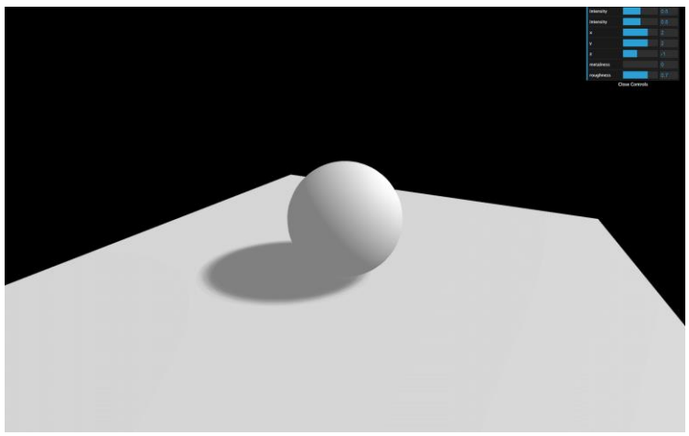
```
directionalLightCameraHelper.visible = false
```



Blur

You can control the shadow blur with the **radius** property:

```
directionalLight.shadow.radius = 10
```



This technique doesn't use the proximity of the camera with the object. It's just a general and cheap blur.

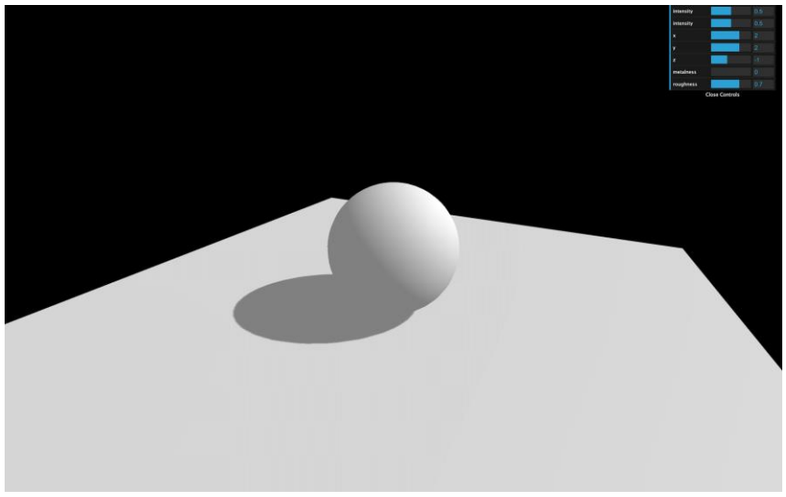
Shadow map algorithm

Different types of algorithms can be applied to shadow maps:

- **THREE.BasicShadowMap:** Very performant but lousy quality
- **THREE.PCFShadowMap:** Less performant but smoother edges
- **THREE.PCFSoftShadowMap:** Less performant but even softer edges
- **THREE.VSMShadowMap:** Less performant, more constraints, can have unexpected results

To change it, update the **renderer.shadowMap.type** property. The default is **THREE.PCFShadowMap** but you can use **THREE.PCFSoftShadowMap** for better quality.

```
renderer.shadowMap.type = THREE.PCFSoftShadowMap
```



Be aware that the radius property doesn't work with **THREE.PCFSoftShadowMap**. You'll have to choose.

SpotLight

Let's try to add a SpotLight like we did in the **Lights** lesson and add the **castShadow** property to **true**. Don't forget to add the **target** property to the **scene**.

We will also add a camera helper:

```
// Spot light
const spotLight = new THREE.SpotLight(0xffffff, 0.4, 10, Math.PI * 0.3)

spotLight.castShadow = true

spotLight.position.set(0, 2, 2)
scene.add(spotLight)
scene.add(spotLight.target)

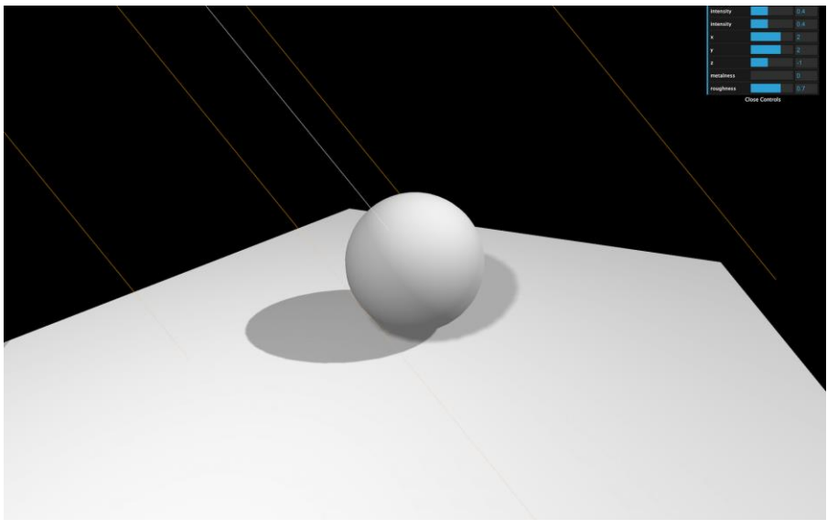
const spotLightCameraHelper = new
THREE.CameraHelper(spotLight.shadow.camera)
scene.add(spotLightCameraHelper)
```

You can reduce the other lights intensity if the scene is too bright:

```
const ambientLight = new THREE.AmbientLight(0xffffff, 0.4)

// ...

const directionalLight = new THREE.DirectionalLight(0xffffff, 0.4)
```

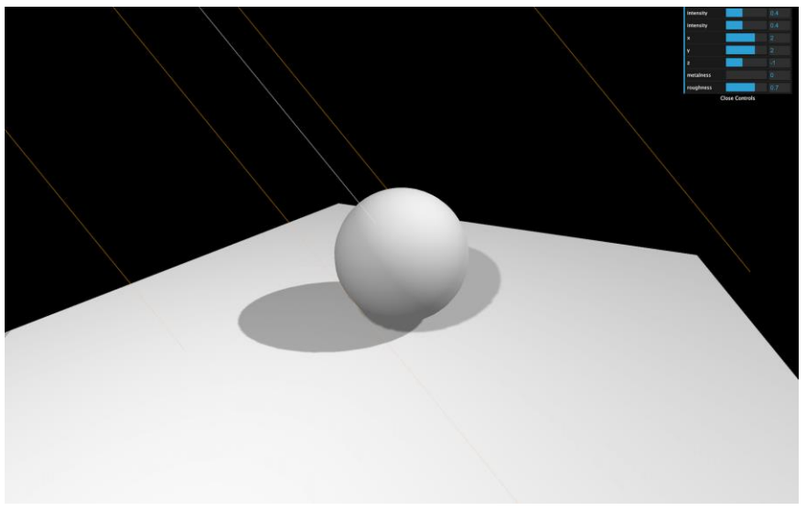


As you can see, shadows don't merge nicely. They are handled independently, and, unfortunately, there is not much to do about it.

But we can improve the shadow quality using the same techniques that we used for the directional light.

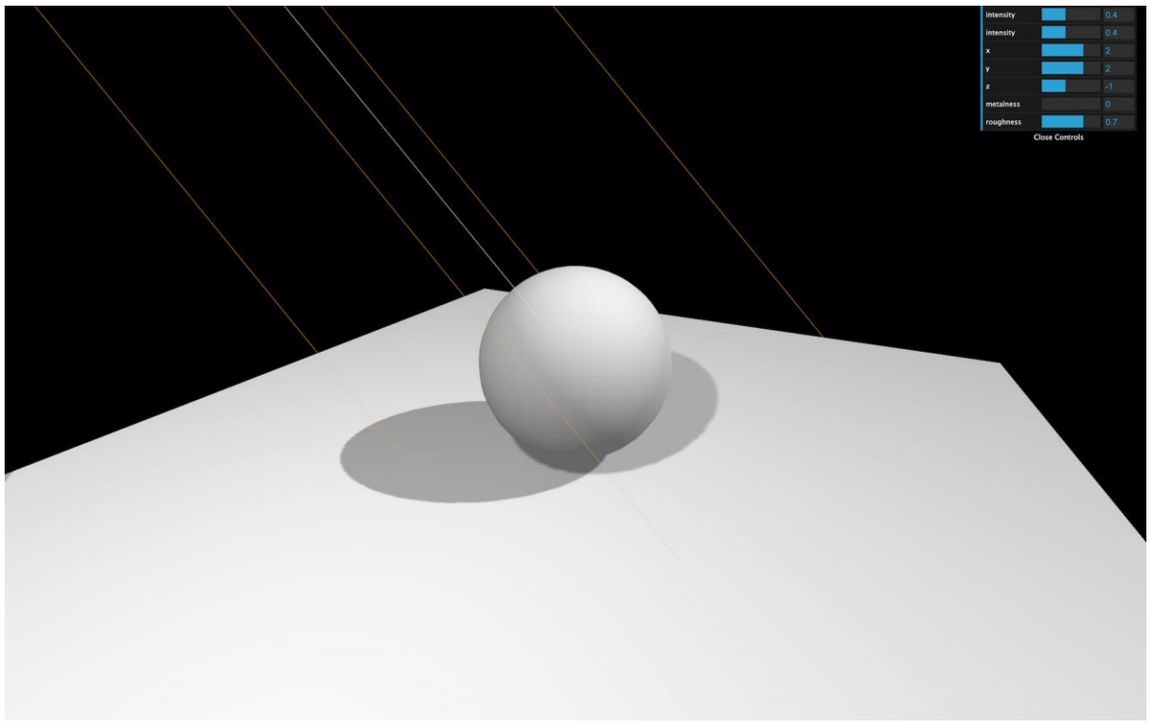
Change the **shadow.mapSize**:

```
spotLight.shadow.mapSize.width = 1024
spotLight.shadow.mapSize.height = 1024
```



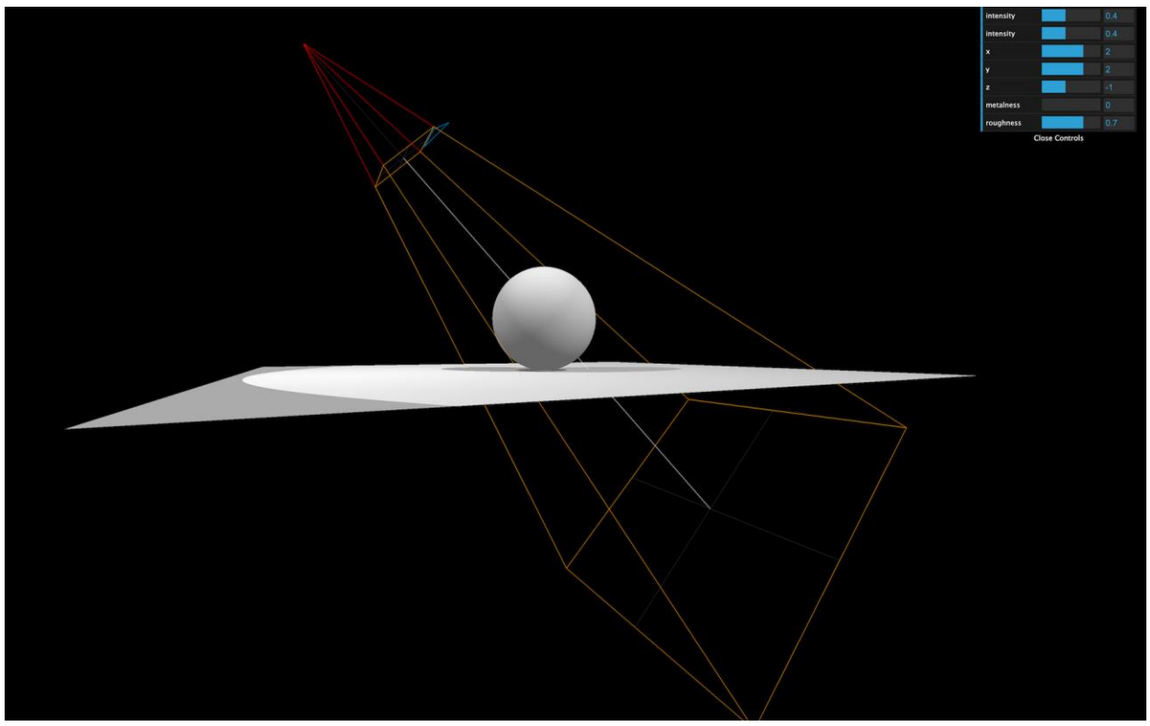
Because we are now using a SpotLight, internally, Three.js is using a PerspectiveCamera. That means that instead of the **top**, **right**, **bottom**, and **left** properties, we must change the **fov** property. Try to find an angle as small as possible without having the shadows cropped:

```
spotLight.shadow.camera.fov = 30
```



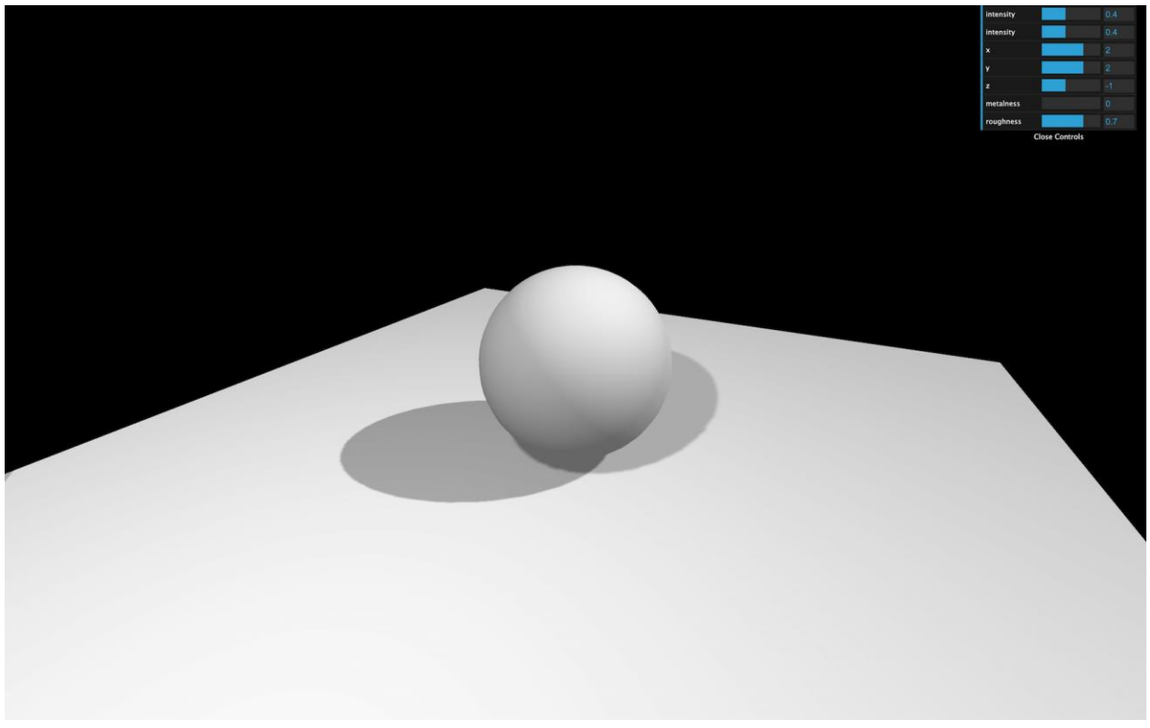
Change the **near** and **far** values:

```
spotLight.shadow.camera.near = 1
spotLight.shadow.camera.far = 6
```



You can hide the camera helper once you're done:

```
spotLightCameraHelper.visible = false
```



PointLight

Let's try the last light supporting shadows, the PointLight:

```
// Point light
const pointLight = new THREE.PointLight(0xffffff, 0.3)

pointLight.castShadow = true

pointLight.position.set(- 1, 1, 0)
scene.add(pointLight)

const pointLightCameraHelper = new
THREE.CameraHelper(pointLight.shadow.camera)
scene.add(pointLightCameraHelper)
```

You can reduce the other lights intensity if the scene is too bright:

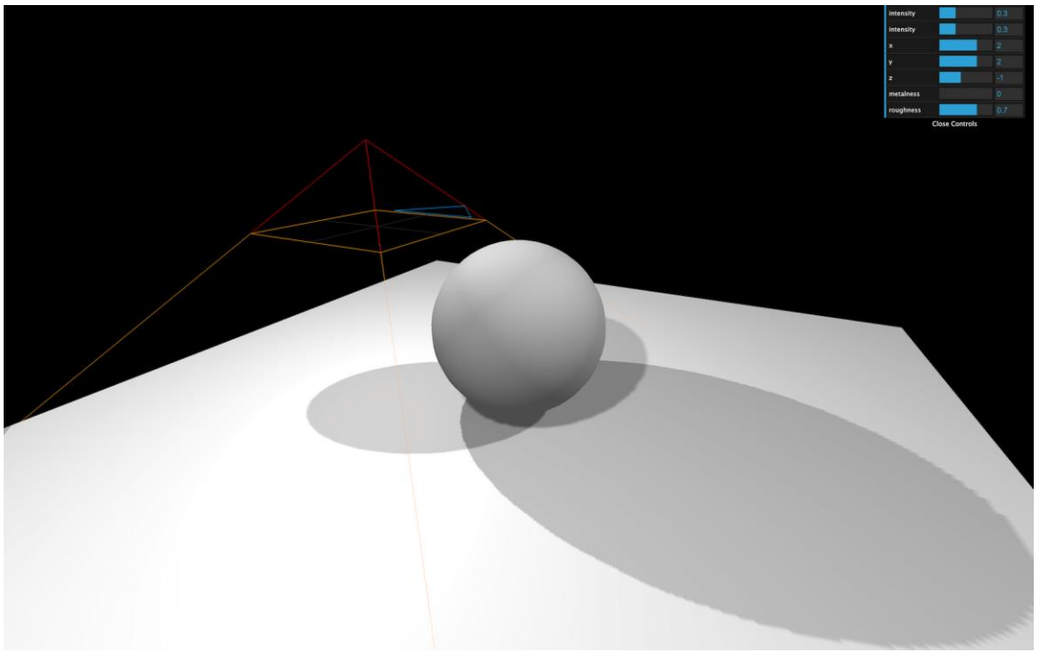
```
const ambientLight = new THREE.AmbientLight(0xffffff, 0.3)

// ...

const directionalLight = new THREE.DirectionalLight(0xffffff, 0.3)

// ...

const spotLight = new THREE.SpotLight(0xffffff, 0.3, 10, Math.PI * 0.3)
```



As you can see, the camera helper is a PerspectiveCamera (like for the SpotLight) but facing downward. That is due to how Three.js handles shadow maps for the PointLight.

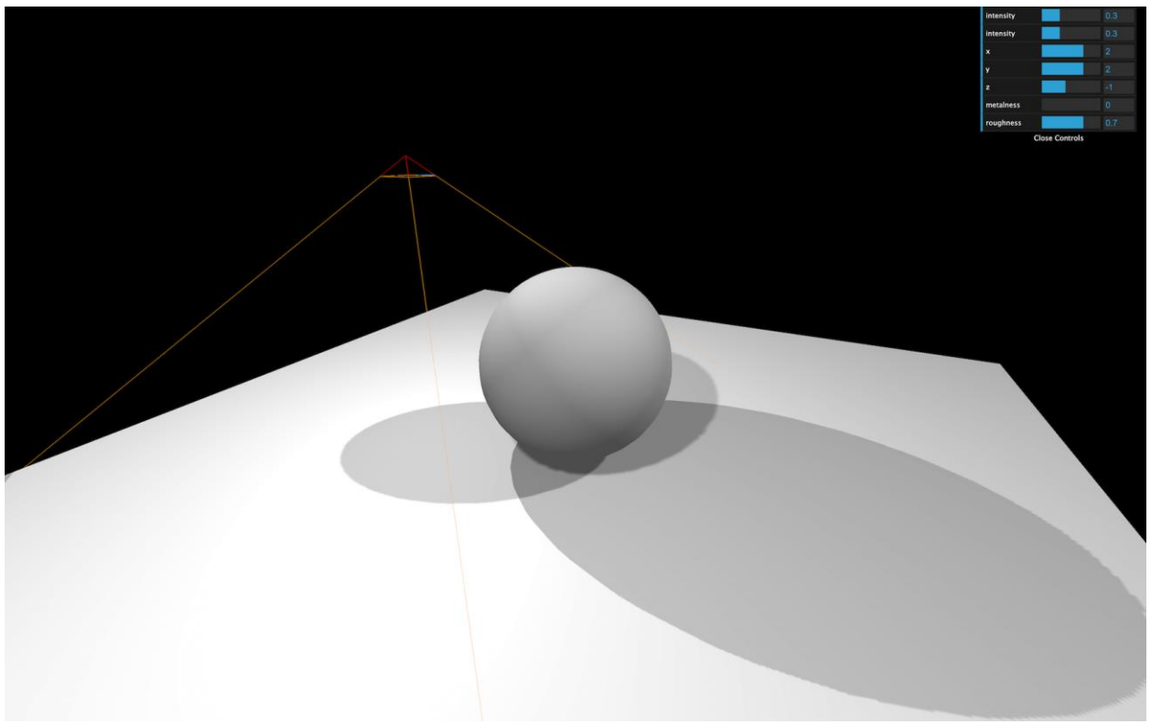
Because the point light illuminates in every direction, Three.js will have to render each of the 6 directions to create a cube shadow map. The camera helper you see is the camera's position in the last of those 6 renders (which is downward).

Doing all those renders can generate performance issues. Try to avoid having too much PointLight with shadows enabled.

The only properties you can tweak here are the **mapSize**, **near** and **far**:

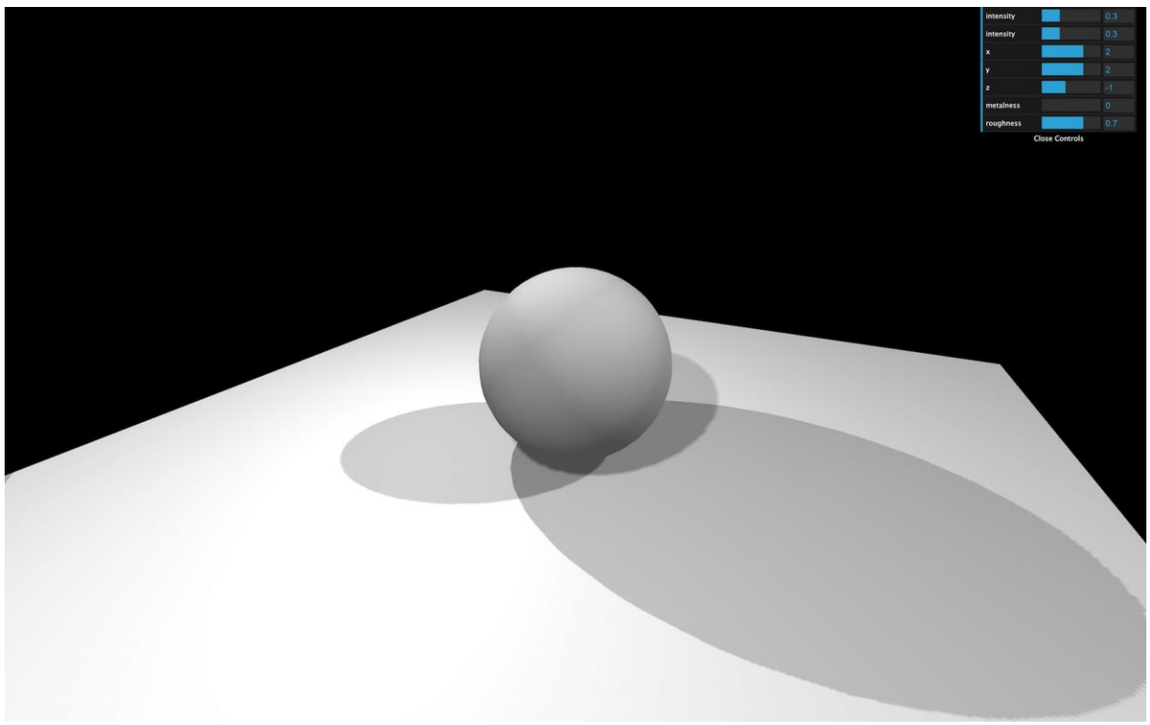
```
pointLight.shadow.mapSize.width = 1024
pointLight.shadow.mapSize.height = 1024

pointLight.shadow.camera.near = 0.1
pointLight.shadow.camera.far = 5
```



You can hide the camera helper once you're done:

```
pointLightCameraHelper.visible = false
```



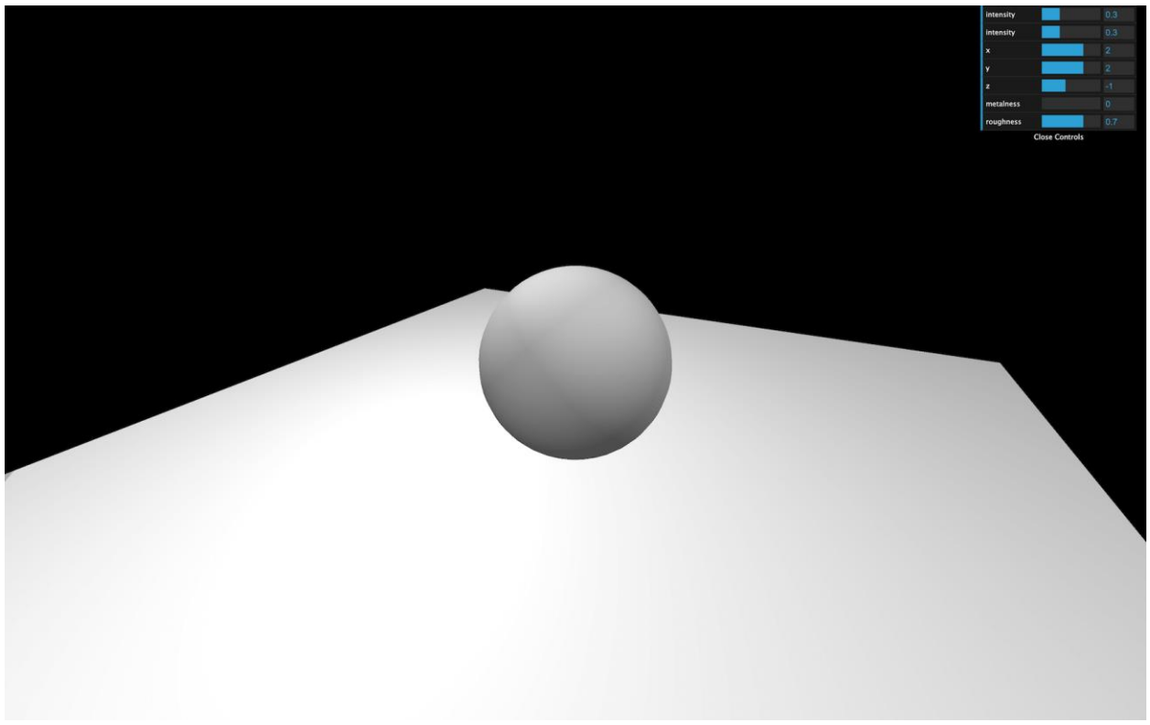
Baking shadows

Three.js shadows can be very useful if the scene is simple, but it might otherwise become messy.

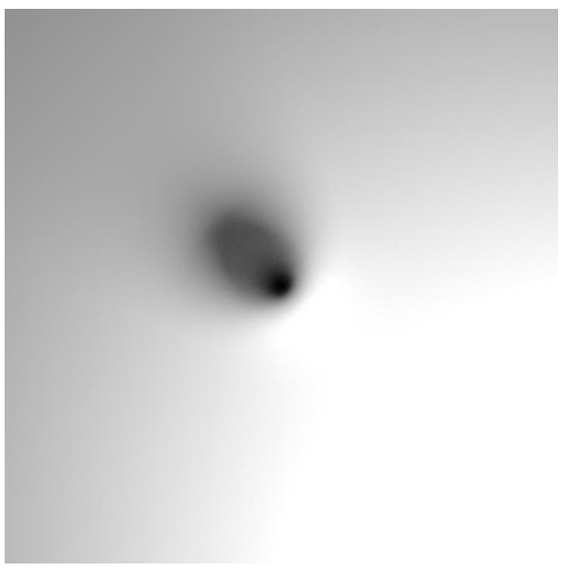
A good alternative is baked shadows. We talk about baked lights in the previous lesson and it is exactly the same thing. Shadows are integrated into textures that we apply on materials.

Instead of commenting all the shadows related lines of code, we can simply deactivate them in the renderer and on each light:

```
directionalLight.castShadow = false
// ...
spotLight.castShadow = false
// ...
pointLight.castShadow = false
// ...
renderer.shadowMap.enabled = false
```



Now we can load a shadow texture located in **/static/textures/bakedShadow.jpg** using the classic TextureLoader.

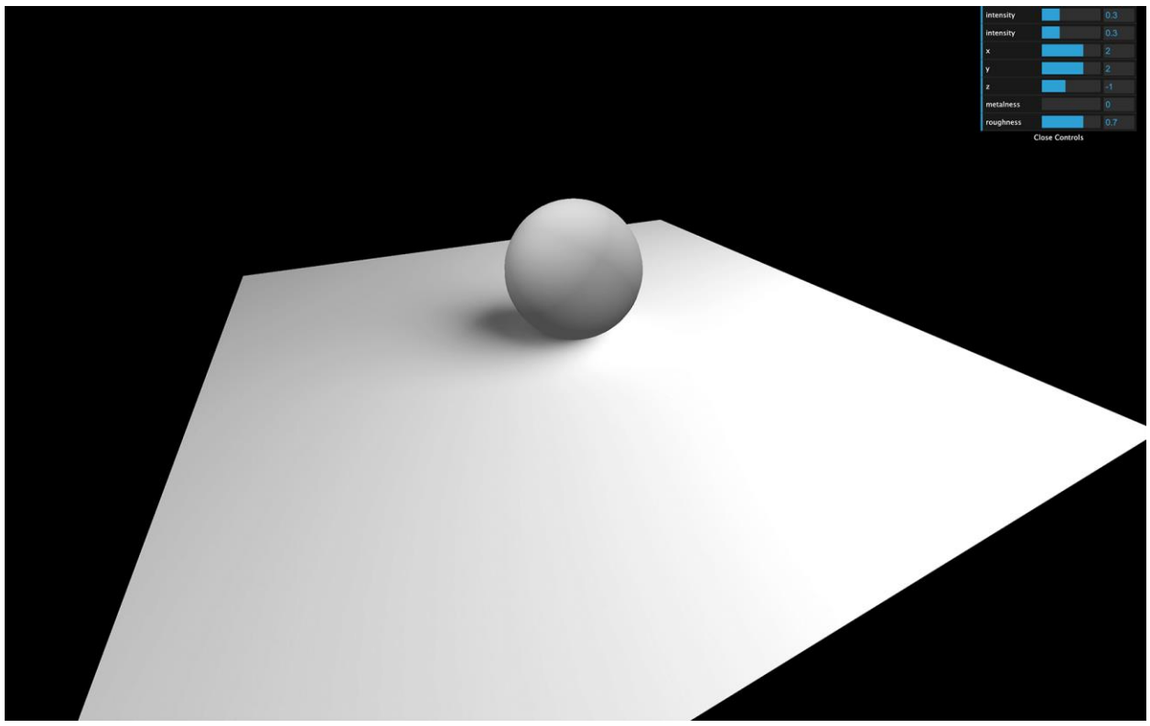


Add the following code before creating the objects and lights:

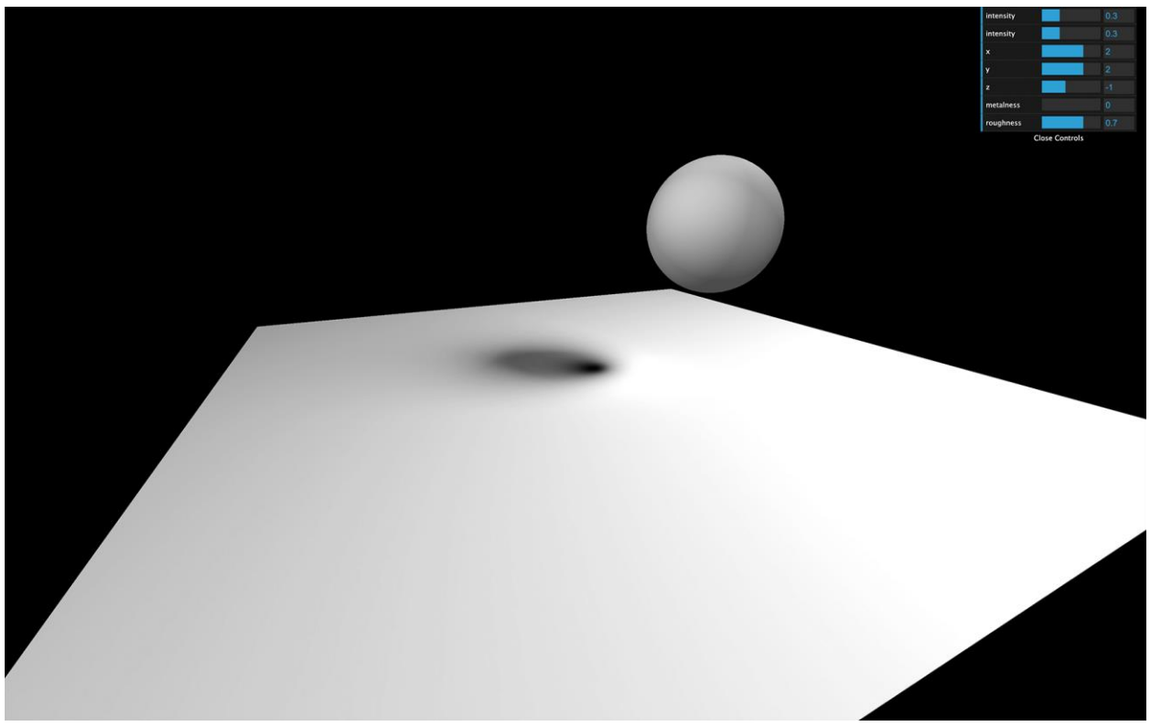
```
/**
 * Textures
 */
const textureLoader = new THREE.TextureLoader()
const bakedShadow = textureLoader.load('/textures/bakedShadow.jpg')
```

And finally, instead of using a MeshStandardMaterial on the plane, we'll use a simple MeshBasicMaterial with the **bakedShadow** as **map**:

```
const plane = new THREE.Mesh(
    new THREE.PlaneGeometry(5, 5),
    new THREE.MeshBasicMaterial({
        map: bakedShadow
    })
)
```

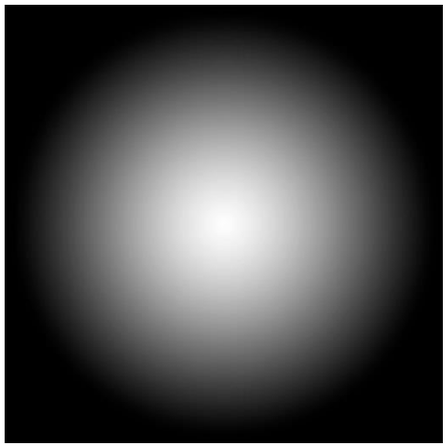


You should see a nice blurred, and realistic fake shadow. The main problem is that it's not dynamic, and if the sphere or the lights moves, the shadows won't.



Baking shadows alternative

A less realistic but more dynamic solution would be to use a more simple shadow under the sphere and slightly above the plane.



The texture is a simple halo. The white part will be visible and the black part will be invisible.

Then, we move that shadow with the sphere.

First, let's remove the previous baked shadow by putting back the MeshStandardMaterial on the plane:

```
const plane = new THREE.Mesh(
    new THREE.PlaneGeometry(5, 5),
    material
)
```

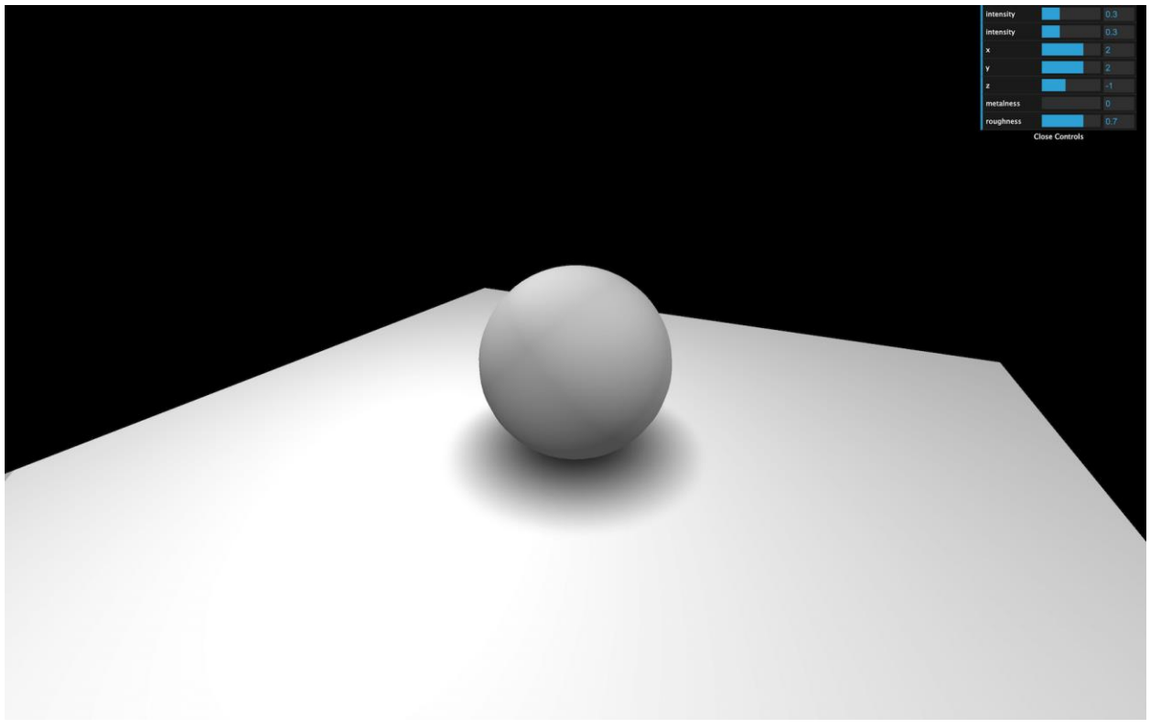
Then, we can load a basic shadow texture located in **/static/textures/simpleShadow.jpg**.

```
const simpleShadow = textureLoader.load('/textures/simpleShadow.jpg')
```

We can create the shadow by using a simple plane that we rotate and place slightly above the floor. The material must be black but with the shadow texture as the **alphaMap**. Don't forget to change **transparent** to **true**, and to add the mesh to the **scene**:

```
const sphereShadow = new THREE.Mesh(
    new THREE.PlaneGeometry(1.5, 1.5),
    new THREE.MeshBasicMaterial({
        color: 0x000000,
        transparent: true,
        alphaMap: simpleShadow
    })
)
sphereShadow.rotation.x = - Math.PI * 0.5
sphereShadow.position.y = plane.position.y + 0.01

scene.add(sphere, sphereShadow, plane)
```



There you go, a not so realistic but very performant shadow.

If you're going to animate the sphere, you can simply animate the shadow accordingly and change its opacity depending on the elevation of the sphere:

```
const clock = new THREE.Clock()

const tick = () =>
{
    const elapsedTime = clock.getElapsedTime()

    // Update the sphere
    sphere.position.x = Math.cos(elapsedTime) * 1.5
    sphere.position.z = Math.sin(elapsedTime) * 1.5
    sphere.position.y = Math.abs(Math.sin(elapsedTime * 3))

    // Update the shadow
    sphereShadow.position.x = sphere.position.x
    sphereShadow.position.z = sphere.position.z
    sphereShadow.material.opacity = (1 - sphere.position.y) * 0.3
```

```
    // ...
}

tick()
```

Which technique to use

Finding the right solution to handle shadows is up to you. It depends on the project, the performances and the techniques you know. You can also combine them.

CARAGA STATE UNIVERSITY
College of Computing and Information Sciences
Mechatronics Building, CSU-Main Campus, Ampayon, Butuan City, Philippines
Competence Service Uprightness
Your Best Option to Success!
Phone (+63 85) 341-3249
(+63 85) 342-3047
(+63 85) 341-2296
Phone: 341-2786 loc. 123
Email Address: ccis@carsu.edu.ph
URL: https://ccis.carsu.edu.ph/

ACTIVITY TITLE: INTRODUCTION TO THREE.JS (PART 3) HAUNTED HOUSE

COURSE/SUBJECT: *APPLICATIONS DEVELOPMENT AND EMERGING TECHNOLOGIES*
COURSE NO.: *ITE-18*
INSTRUCTOR/PROF: *JAMES EARL D. CUBILLAS, M.Sc., ECT*

Introduction

Let's use what we've learned to create a haunted house. We will only use Three.js primitives as geometries, the textures in the **/static/textures/** folder, and one or two new features.

We will create an elementary house composed of walls, a roof, a door, and some bushes. We will also produce graves in the garden. Instead of visible ghosts made of sheets, we will simply use multicolor lights floating around and passing through the walls and the floor.

Tips for measurements

One beginner mistake we always make when creating something using primitives is using random measures. One unit in Three.js can mean anything you want.

Suppose you are creating a considerable landscape to fly above. In that case, you might think of one unit as one kilometer. If you are building a house, you might think of one unit as one meter; if you are making a marble game, you might think of one unit as one centimeter.

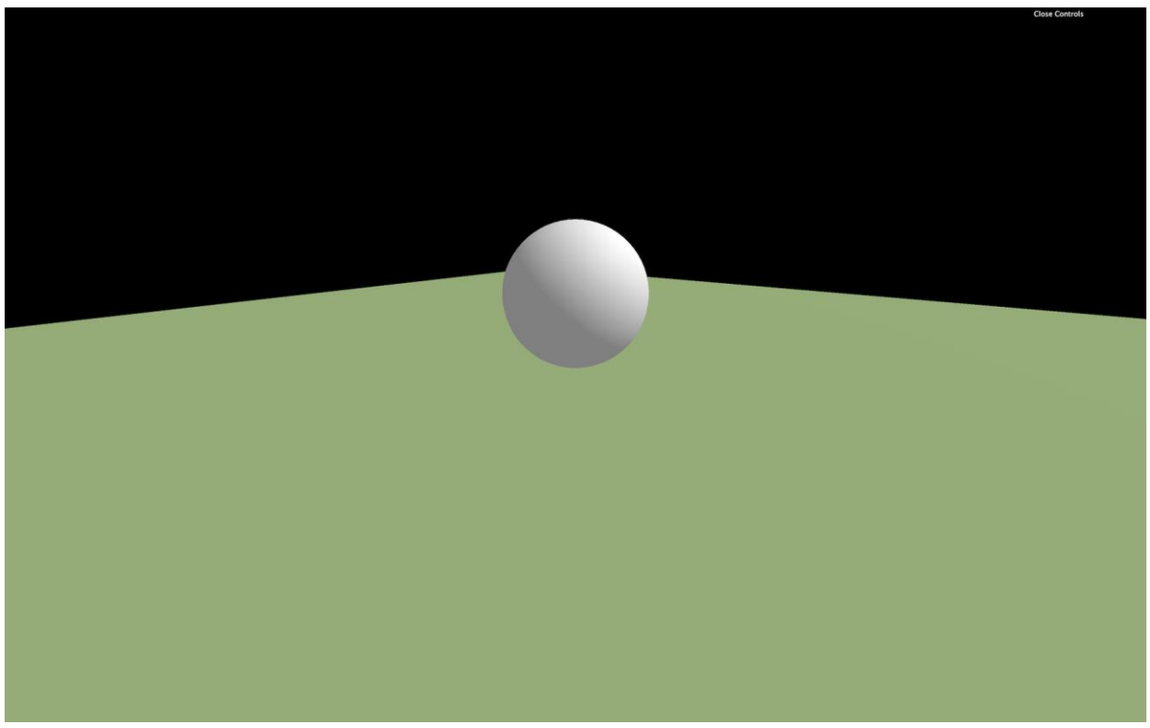
Having a specific unit ratio will help you create geometries. Let's say you want to make the door. You know a door is slightly taller than you, so it should reach around 2 meters.

Those using imperial units will have to do the conversion.

Setup

The starter is composed of a floor, a sphere, some lights (way too intense for a haunted house), and shadows that aren't even working.

We will have to create the house all by ourselves, tweak the current lights for a better ambiance, and add the shadows and the rest.



The house

First, let's remove the sphere and create a tiny house. We can leave the floor.

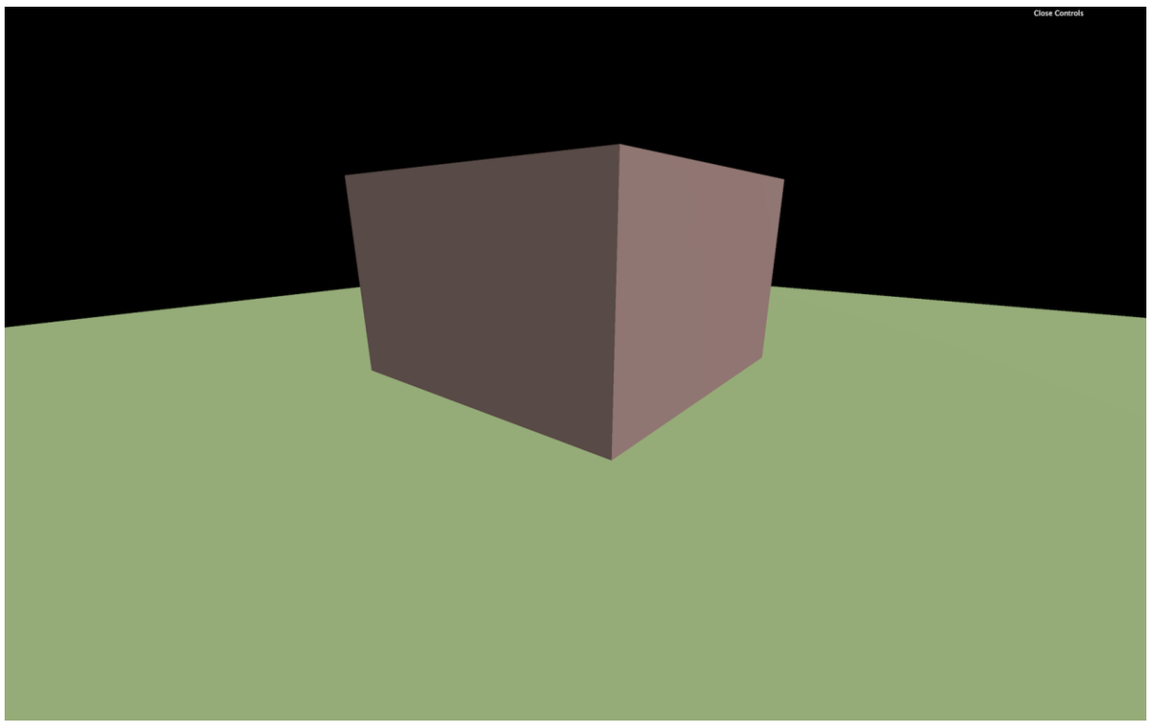


Instead of putting every object composing that house in the scene, we will first create a container just in case we want to move or scale the whole thing:

```
// House container
const house = new THREE.Group()
scene.add(house)
```

Then we can create the walls with a simple cube and add it to the **house**. Don't forget to move the walls up on the **y** axis; otherwise it will be half inside the floor:

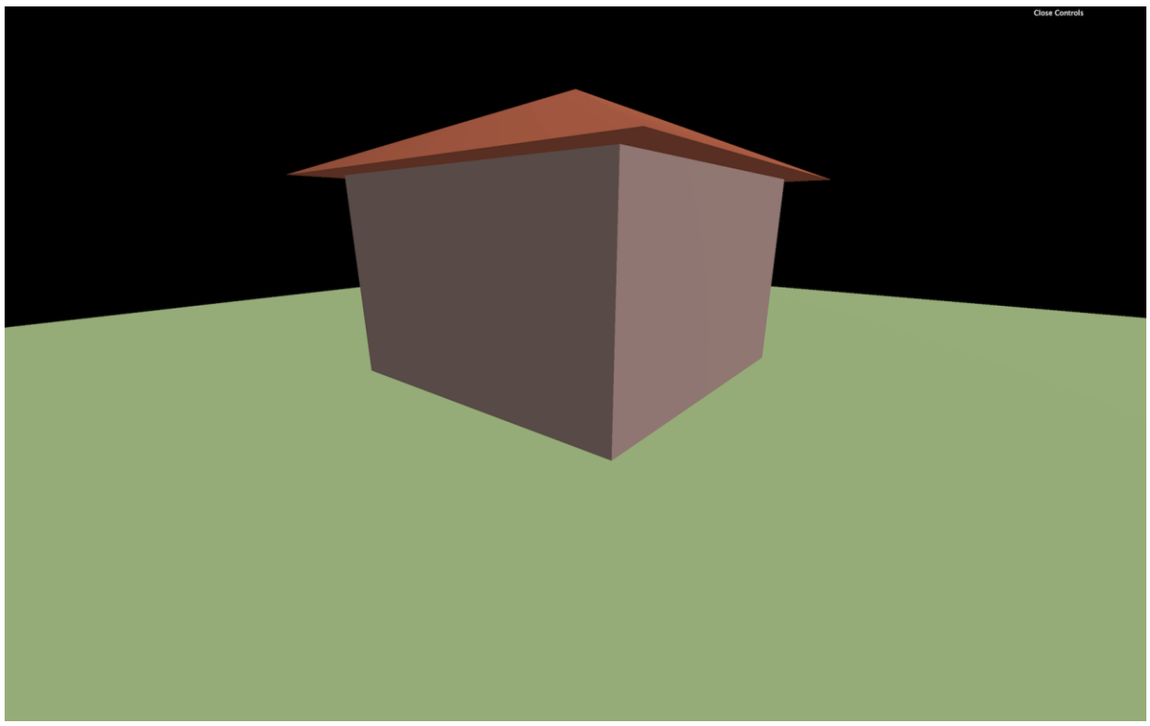
```
const walls = new THREE.Mesh(
    new THREE.BoxGeometry(4, 2.5, 4),
    new THREE.MeshStandardMaterial({ color: '#ac8e82' })
)
walls.position.y = 1.25
house.add(walls)
```



We chose **2.5** for the height because it would seem like a normal height for the ceiling. We also chose **'#ac8e82'** for the color, but it's temporary, and we will replace those colors with textures later.

For the roof, we want to make a pyramid shape. The problem is that Three.js doesn't have this kind of geometry. But if you start from a cone and reduce the number of sides to **4**, you'll get a pyramid. Sometimes you just have to deflect basic features usages:

```
// Roof
const roof = new THREE.Mesh(
    new THREE.ConeGeometry(3.5, 1, 4),
    new THREE.MeshStandardMaterial({ color: '#b35f45' })
)
roof.rotation.y = Math.PI * 0.25
roof.position.y = 2.5 + 0.5
house.add(roof)
```

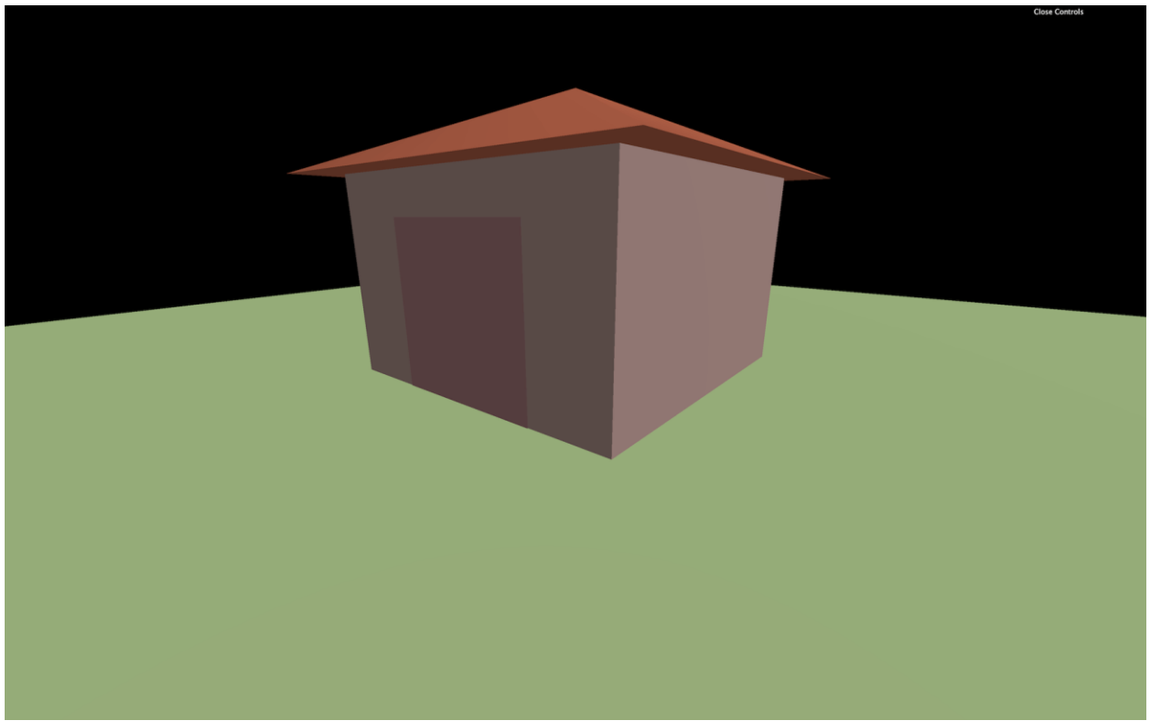


Finding the right position and the right rotation might be a little hard. Take your time, try to figure out the logic behind the values, and don't forget that **Math.PI** is your friend.

As you can see, we left **2.5 + 0.5**. We could have written **3** but it's sometime better to visualize the logic behind the value. **2.5**, because the roof walls are **2.5** units high and **0.5** because the cone is **1** unit high (and we need to move it up to half its height).

We will use a simple plane for the door as we are going to use the beautiful door texture we used in a previous lesson.

```
// Door
const door = new THREE.Mesh(
    new THREE.PlaneGeometry(2, 2),
    new THREE.MeshStandardMaterial({ color: '#aa7b7b' })
)
door.position.y = 1
door.position.z = 2 + 0.01
house.add(door)
```



We don't know yet if the plane has the right size, but we can fix that later when we have the textures working.

As you can see, we move the door on the **z** axis to stick it to the wall but we also added **0.01** units. If you don't add this small value, you'll have a bug we already saw in a previous lesson called z-fighting. Z-fighting happens when you have two faces in the same position (or very close). The GPU doesn't know which one is closer than the other, and you get some strange visual pixel fighting.

Let's add some bushes. Instead of creating one geometry for each bush, we will create only one, and all meshes will share it. The result will be visually the same, but we'll get a performance improvement. We can do the same with the material.

```
// Bushes
const bushGeometry = new THREE.SphereGeometry(1, 16, 16)
const bushMaterial = new THREE.MeshStandardMaterial({ color: '#89c854' })

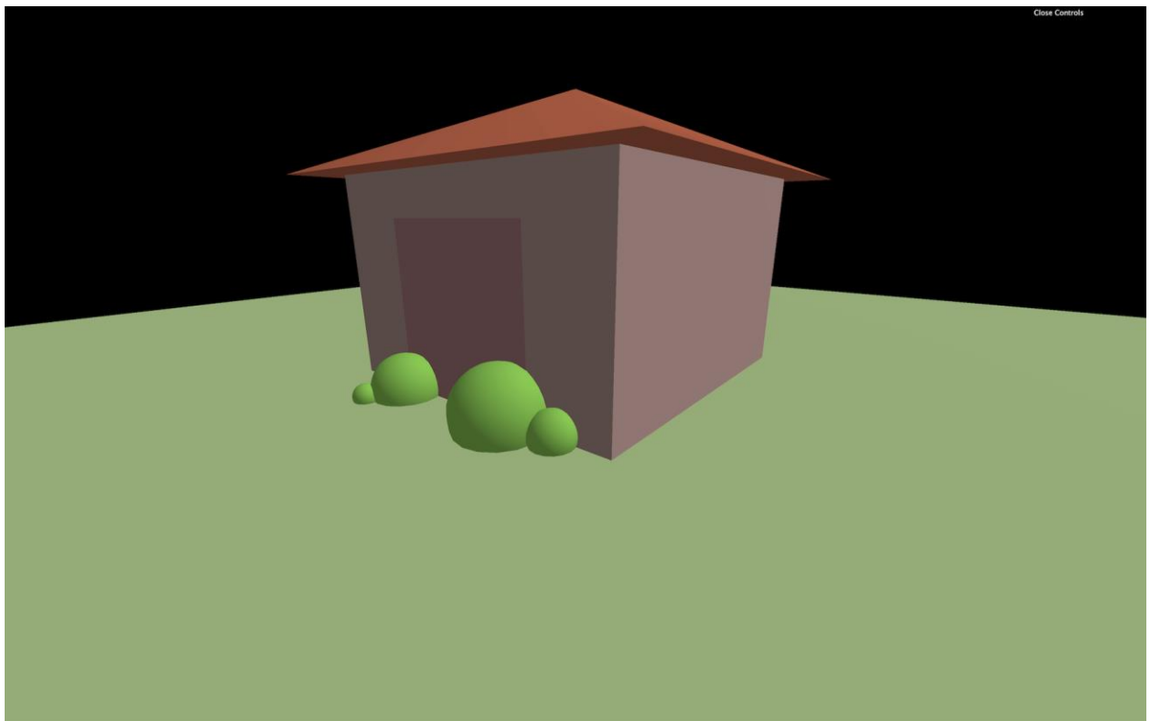
const bush1 = new THREE.Mesh(bushGeometry, bushMaterial)
bush1.scale.set(0.5, 0.5, 0.5)
bush1.position.set(0.8, 0.2, 2.2)

const bush2 = new THREE.Mesh(bushGeometry, bushMaterial)
bush2.scale.set(0.25, 0.25, 0.25)
bush2.position.set(1.4, 0.1, 2.1)

const bush3 = new THREE.Mesh(bushGeometry, bushMaterial)
bush3.scale.set(0.4, 0.4, 0.4)
bush3.position.set(- 0.8, 0.1, 2.2)

const bush4 = new THREE.Mesh(bushGeometry, bushMaterial)
bush4.scale.set(0.15, 0.15, 0.15)
bush4.position.set(- 1, 0.05, 2.6)

house.add(bush1, bush2, bush3, bush4)
```



It does take too long to place and scale all these objects directly in the code. In a later lesson, we will learn how to use a 3D software to create all of this.

We won't add too many details to the house because we must move forward, but feel free to pause and add anything you want like low walls, an alley, windows, a chimney, rocks, etc.

The graves

Instead of placing each grave manually, we are going to create and place them procedurally.

The idea is to place the graves randomly on a circle around the house.

First, let's create a container just in case:

```
// Graves
const graves = new THREE.Group()
scene.add(graves)
```

Like in the **3D Text** lesson where we created multiple donuts with one geometry and one material, we are going to create one BoxGeometry and one MeshStandardMaterial that will be shared amongst every graves:

```
const graveGeometry = new THREE.BoxGeometry(0.6, 0.8, 0.2)
const graveMaterial = new THREE.MeshStandardMaterial({ color: '#b2b6b1' })
```

Finally, let's loop and do some mathematics to position a bunch of graves around the house.

We are going to create a random angle on a circle. Remember that a full revolution is 2 times π. Then we are going to use that angle on both a **sin(...)** and a **cos(...)**. This is how you position things on a circle when you have the angle. And finally we also multiply those **sin(...)** and **cos(...)** results by a random value because we don't want the graves to be positioned on a perfect circle.

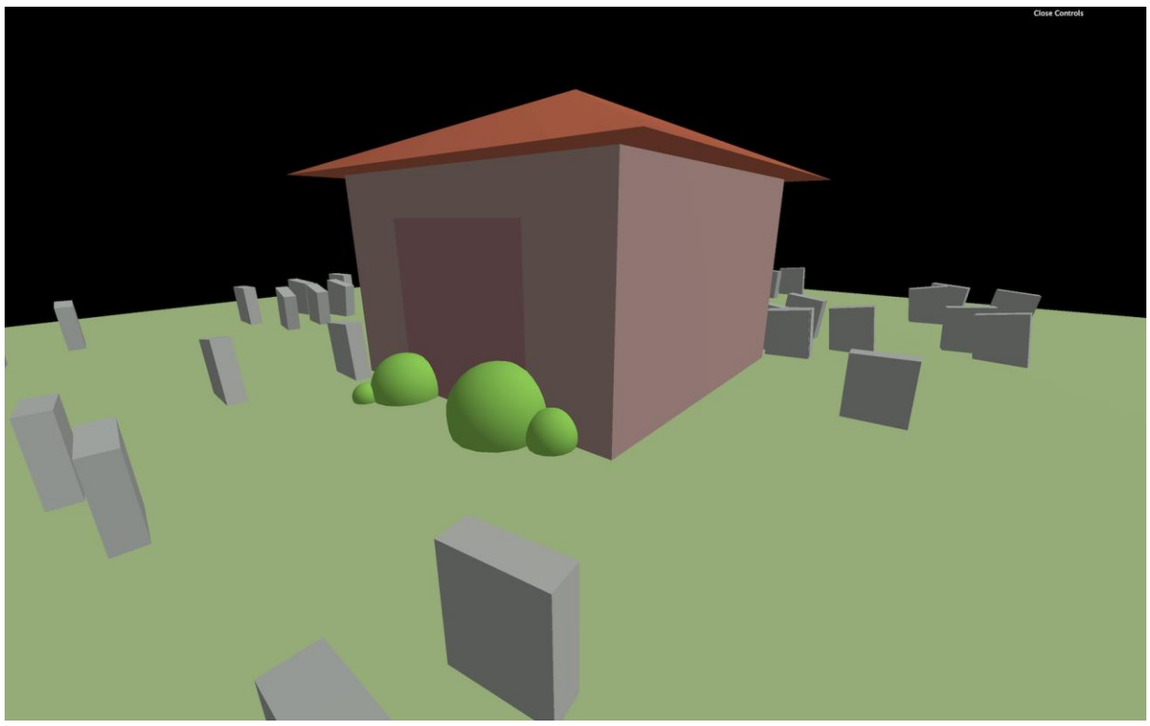
```
for(let i = 0; i < 50; i++)
{
    const angle = Math.random() * Math.PI * 2 // Random angle
    const radius = 3 + Math.random() * 6      // Random radius
    const x = Math.cos(angle) * radius        // Get the x position using
cosinus
    const z = Math.sin(angle) * radius        // Get the z position using
sinus

    // Create the mesh
    const grave = new THREE.Mesh(graveGeometry, graveMaterial)

    // Position
    grave.position.set(x, 0.3, z)

    // Rotation
    grave.rotation.z = (Math.random() - 0.5) * 0.4
    grave.rotation.y = (Math.random() - 0.5) * 0.4

    // Add to the graves container
    graves.add(grave)
}
```



Lights

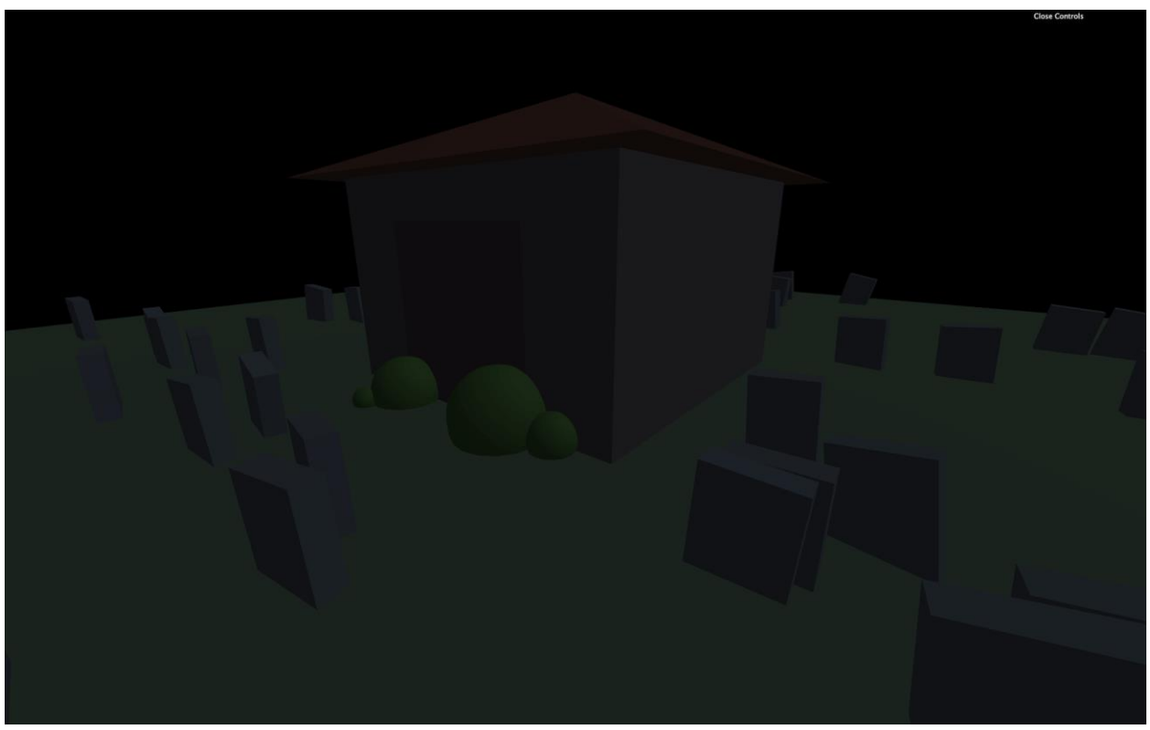
We have a pretty cool scene, but it's not that scary yet.

First, let's dim the ambient and moon lights and give those a more blue-ish color:

```
const ambientLight = new THREE.AmbientLight('#b9d5ff', 0.12)

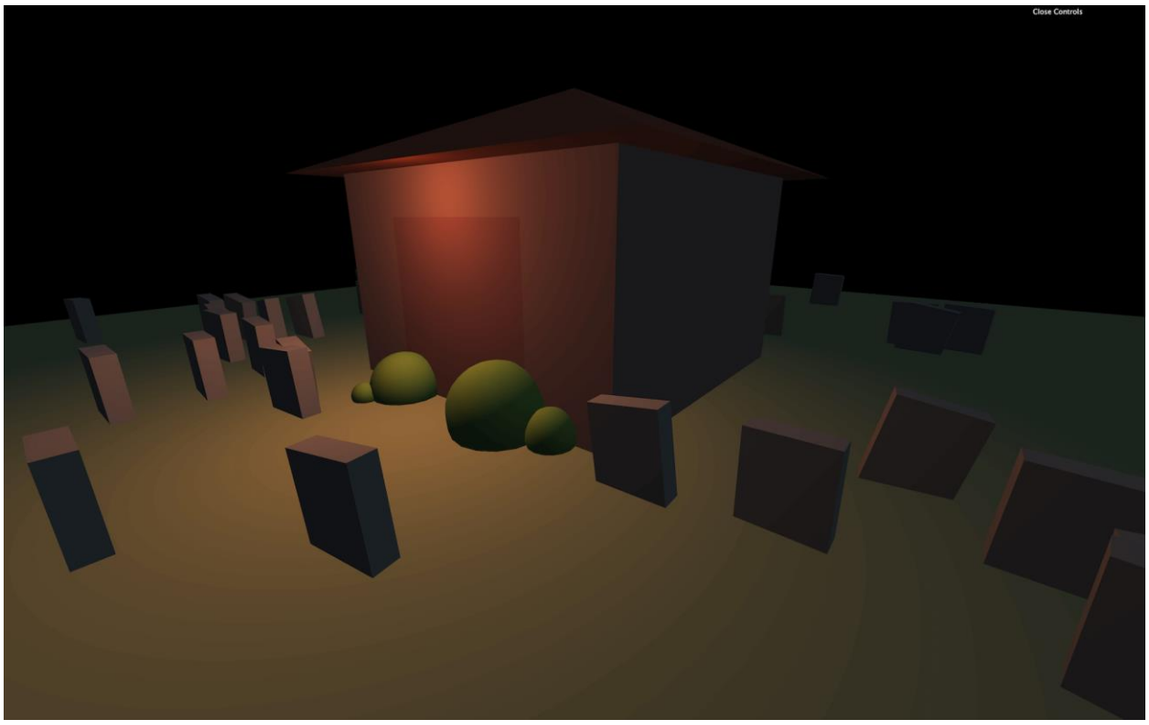
// ...

const moonLight = new THREE.DirectionalLight('#b9d5ff', 0.12)
```



We can't see much right now. Let's also add a warm PointLight above the door. Instead of adding this light to the scene, we can add it to the house:

```
// Door light
const doorLight = new THREE.PointLight('#ff7d46', 1, 7)
doorLight.position.set(0, 2.2, 2.7)
house.add(doorLight)
```



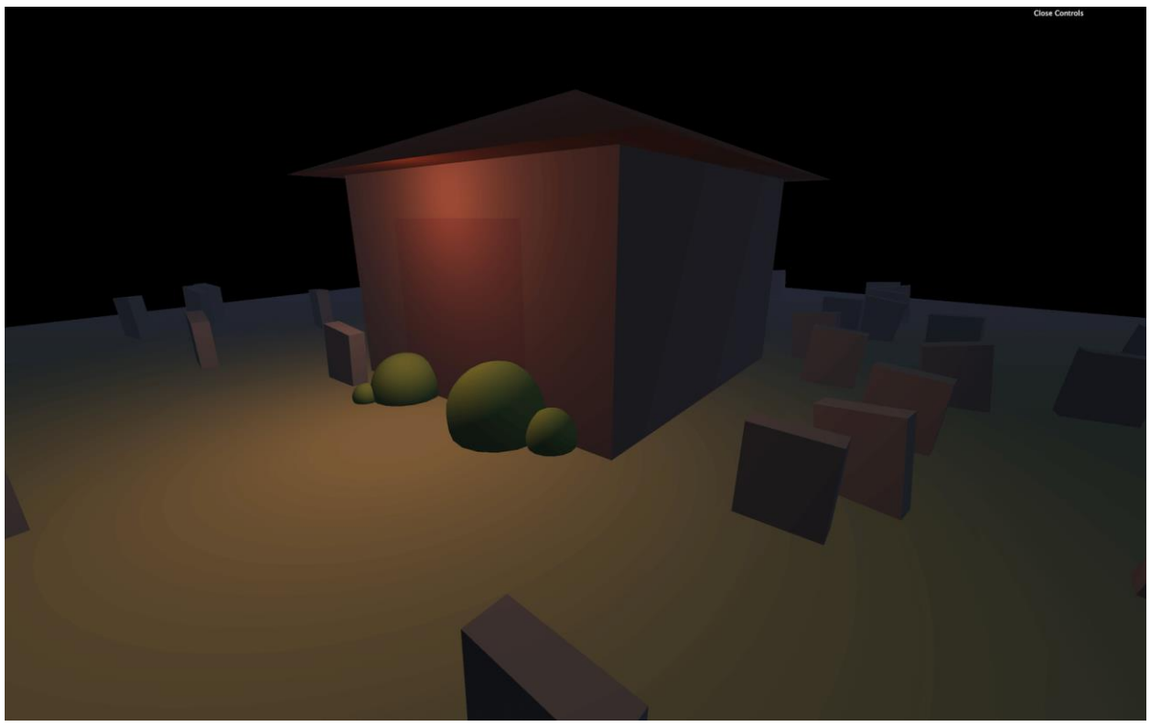
Fog

In horror movies, they always use fog. The good news is that Three.js supports it already with the Fog class.

The first parameter is the **color**, the second parameter is the **near** (how far from the camera does the fog start), and the third parameter is the **far** (how far from the camera will the fog be fully opaque).

To activate the fog, add the **fog** property to the **scene**:

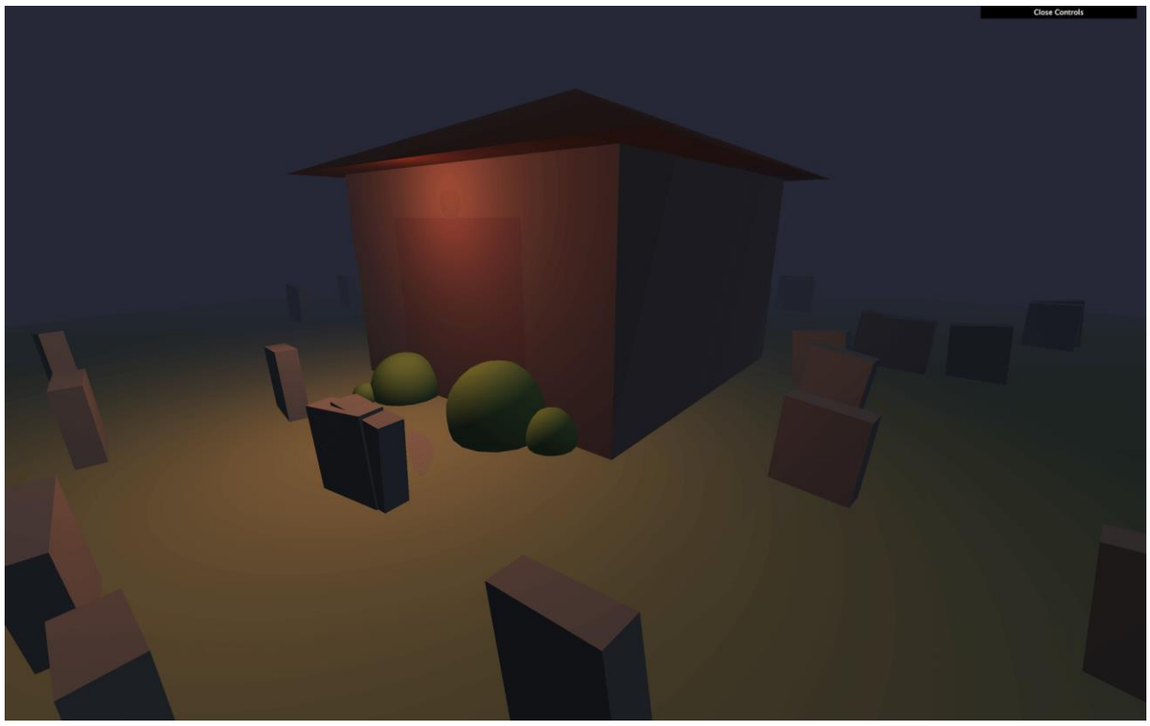
```
/**
 * Fog
 */
const fog = new THREE.Fog('#262837', 1, 15)
scene.fog = fog
```



Not bad, but we can see a clean cut between the graves and the black background.

To fix that, we must change the clear color of the renderer and use the same color as the fog. Do that after instantiating the renderer:

```
renderer.setClearColor('#262837')
```



Here's a slightly scarier scene.

Textures

For even more realism, we can add textures. The **textureLoader** is already in the code.

The door

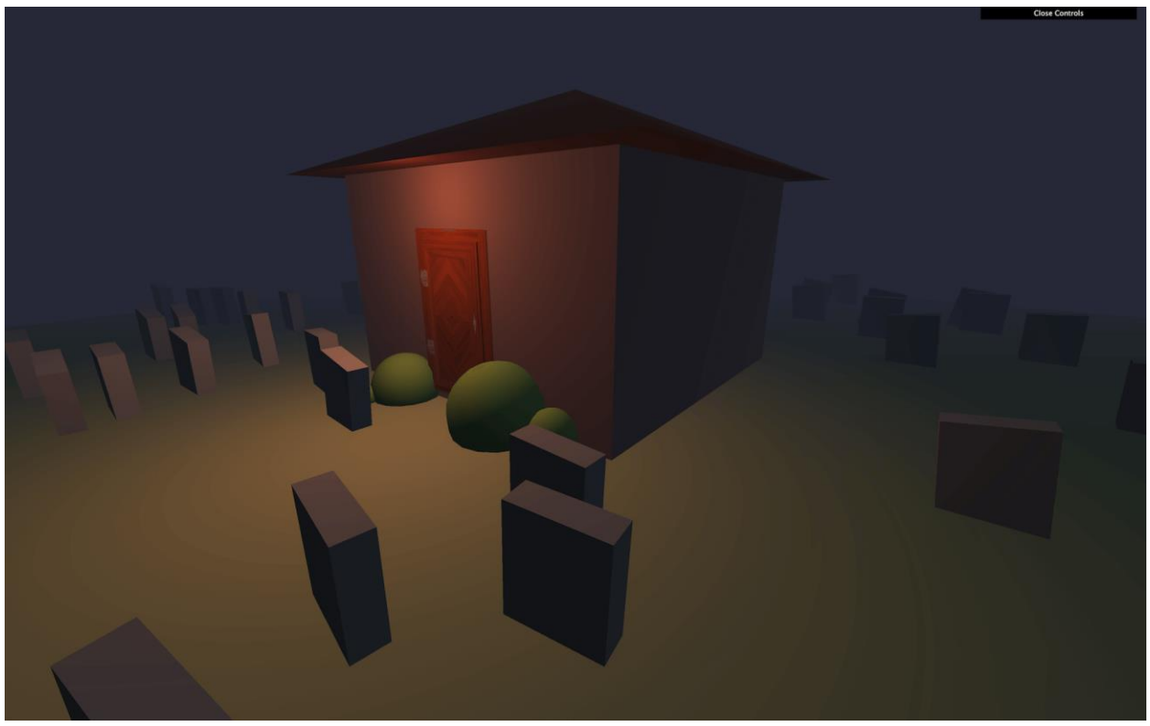
Let's start with something we already know and load all the door textures:

```
const doorColorTexture = textureLoader.load('/textures/door/color.jpg')
const doorAlphaTexture = textureLoader.load('/textures/door/alpha.jpg')
const doorAmbientOcclusionTexture =
textureLoader.load('/textures/door/ambientOcclusion.jpg')
const doorHeightTexture = textureLoader.load('/textures/door/height.jpg')
const doorNormalTexture = textureLoader.load('/textures/door/normal.jpg')
const doorMetalnessTexture =
textureLoader.load('/textures/door/metalness.jpg')
const doorRoughnessTexture =
textureLoader.load('/textures/door/roughness.jpg')
```

Then we can apply all those textures to the door material. Don't forget to add more subdivisions to the PlaneGeometry, so the **displacementMap** has some vertices to move. Also, add the **uv2** attribute to the geometry for the **aoMap** as we did in the **Materials** lesson.

You can access the door's geometry by using **mesh.geometry**:

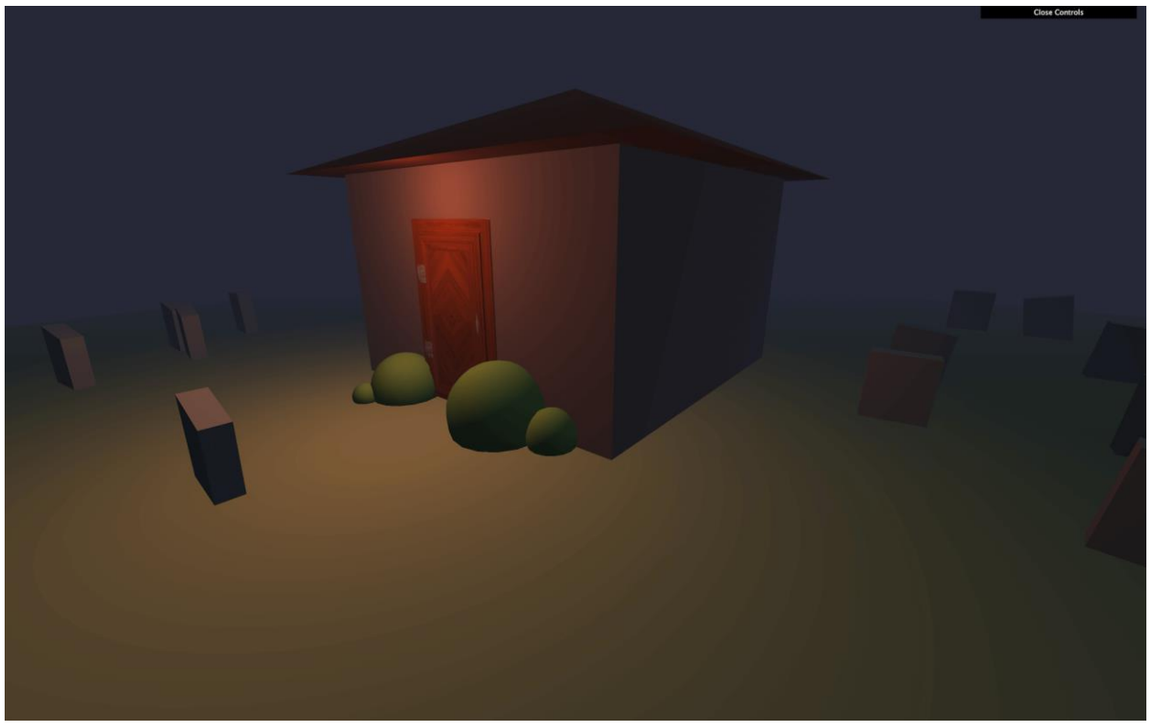
```
const door = new THREE.Mesh(
    new THREE.PlaneGeometry(2, 2, 100, 100),
    new THREE.MeshStandardMaterial({
        map: doorColorTexture,
        transparent: true,
        alphaMap: doorAlphaTexture,
        aoMap: doorAmbientOcclusionTexture,
        displacementMap: doorHeightTexture,
        displacementScale: 0.1,
        normalMap: doorNormalTexture,
        metalnessMap: doorMetalnessTexture,
        roughnessMap: doorRoughnessTexture
    })
)
door.geometry.setAttribute('uv2', new
THREE.Float32BufferAttribute(door.geometry.attributes.uv.array, 2))
```



There you go! That's a more realistic door.

Now that we have the textures, you realize that the door is a little too small. You can simply increase the PlaneGeometry sizes:

```
// ...
    new THREE.PlaneGeometry(2.2, 2.2, 100, 100),
// ...
```



The walls

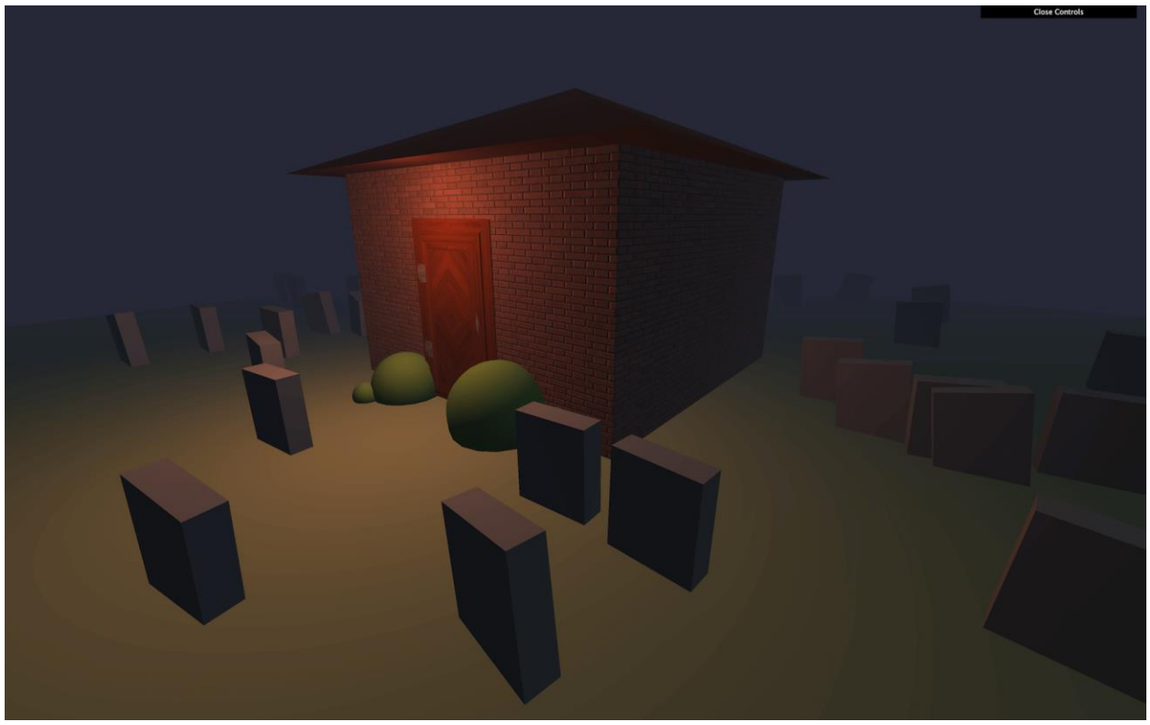
Let's do the same for the walls using the textures on the **/static/textures/bricks/** folder. We don't have as many textures as for the door, but it's not a problem. We don't need an alpha texture, and the wall has no metal in it, so we don't need a metalness texture either.

Load the textures:

```
const bricksColorTexture = textureLoader.load('/textures/bricks/color.jpg')
const bricksAmbientOcclusionTexture =
textureLoader.load('/textures/bricks/ambientOcclusion.jpg')
const bricksNormalTexture =
textureLoader.load('/textures/bricks/normal.jpg')
const bricksRoughnessTexture =
textureLoader.load('/textures/bricks/roughness.jpg')
```

Then we can update our MeshStandardMaterial for the wall. Don't forget to remove the **color** and add the **uv2** attribute for the ambient occlusion.

```
const walls = new THREE.Mesh(
    new THREE.BoxGeometry(4, 2.5, 4),
    new THREE.MeshStandardMaterial({
        map: bricksColorTexture,
        aoMap: bricksAmbientOcclusionTexture,
        normalMap: bricksNormalTexture,
        roughnessMap: bricksRoughnessTexture
    })
)
walls.geometry.setAttribute('uv2', new
THREE.Float32BufferAttribute(walls.geometry.attributes.uv.array, 2))
```



The floor

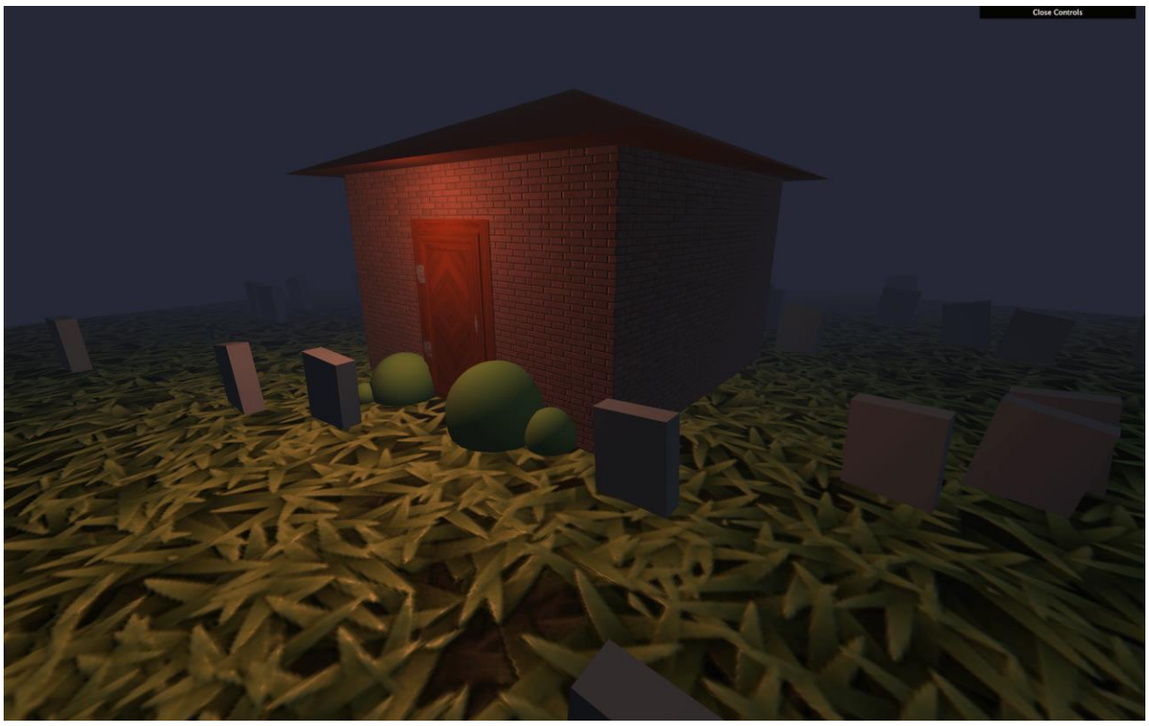
Same deal as for the walls. The grass textures are located in the **/static/textures/grass/** folder.

Load the textures:

```
const grassColorTexture = textureLoader.load('/textures/grass/color.jpg')
const grassAmbientOcclusionTexture =
textureLoader.load('/textures/grass/ambientOcclusion.jpg')
const grassNormalTexture = textureLoader.load('/textures/grass/normal.jpg')
const grassRoughnessTexture =
textureLoader.load('/textures/grass/roughness.jpg')
```

Update the MeshStandardMaterial of the floor and don't forget to remove the **color** and add the **uv2** attribute for the ambient occlusion:

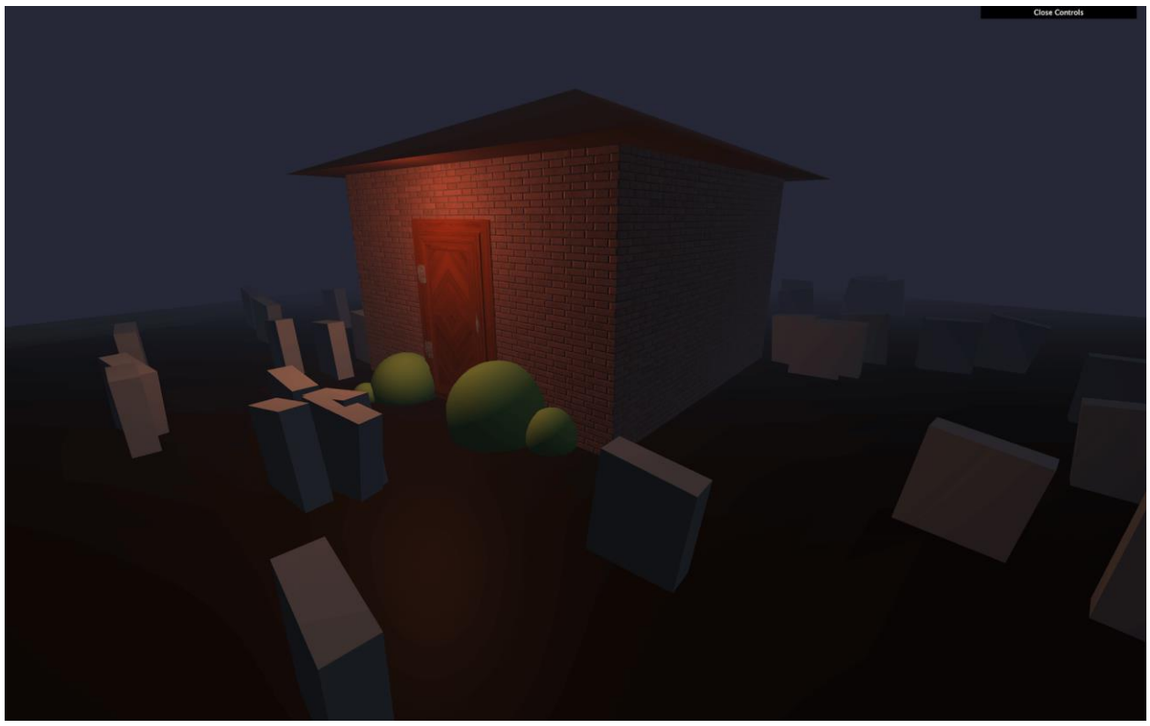
```
const floor = new THREE.Mesh(
    new THREE.PlaneGeometry(20, 20),
    new THREE.MeshStandardMaterial({
        map: grassColorTexture,
        aoMap: grassAmbientOcclusionTexture,
        normalMap: grassNormalTexture,
        roughnessMap: grassRoughnessTexture
    })
)
floor.geometry.setAttribute('uv2', new
THREE.Float32BufferAttribute(floor.geometry.attributes.uv.array, 2))
```



The texture is too large. To fix that, we can simply repeat each grass texture with the repeat property:

```
grassColorTexture.repeat.set(8, 8)
grassAmbientOcclusionTexture.repeat.set(8, 8)
grassNormalTexture.repeat.set(8, 8)
```

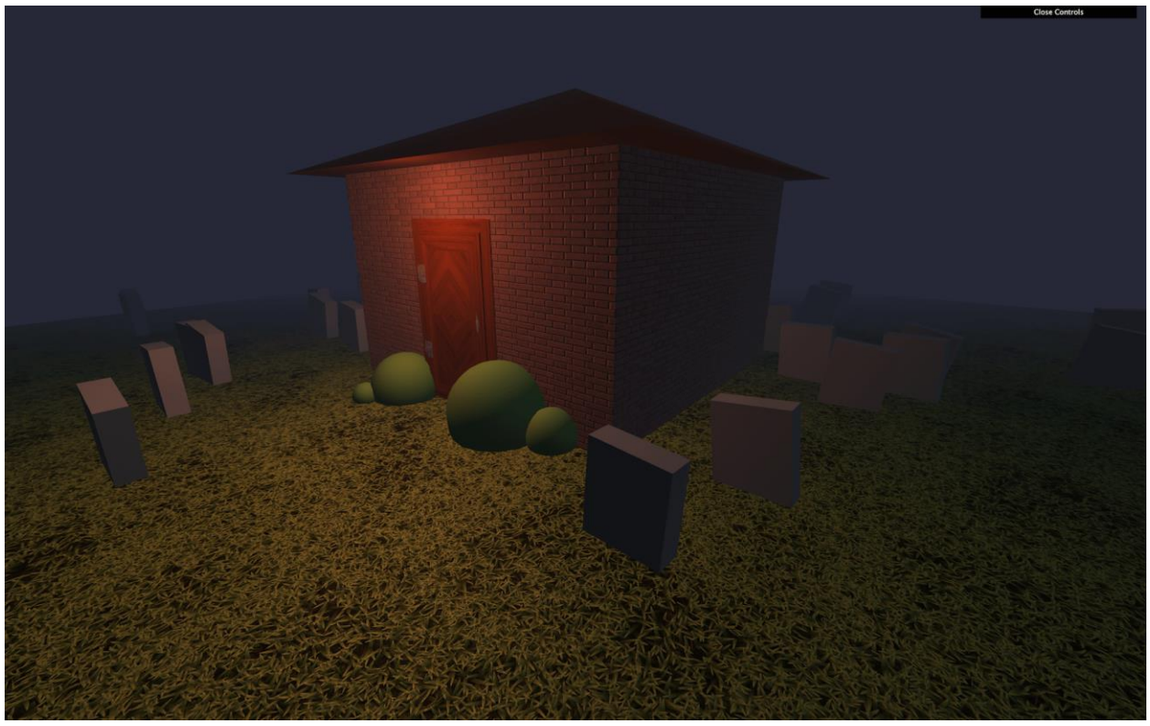
```
grassRoughnessTexture.repeat.set(8, 8)
```



And don't forget to change the **wrapS** and **wrapT** properties to activate the repeat:

```
grassColorTexture.wrapS = THREE.RepeatWrapping
grassAmbientOcclusionTexture.wrapS = THREE.RepeatWrapping
grassNormalTexture.wrapS = THREE.RepeatWrapping
grassRoughnessTexture.wrapS = THREE.RepeatWrapping

grassColorTexture.wrapT = THREE.RepeatWrapping
grassAmbientOcclusionTexture.wrapT = THREE.RepeatWrapping
grassNormalTexture.wrapT = THREE.RepeatWrapping
grassRoughnessTexture.wrapT = THREE.RepeatWrapping
```



Ghosts

For the ghosts, let's keep things simple and do with what we know.

We are going to use simple lights floating around the house and passing through the ground and graves.

```
/**
 * Ghosts
 */
const ghost1 = new THREE.PointLight('#ff00ff', 2, 3)
scene.add(ghost1)

const ghost2 = new THREE.PointLight('#00ffff', 2, 3)
scene.add(ghost2)

const ghost3 = new THREE.PointLight('#ffff00', 2, 3)
scene.add(ghost3)
```

Now we can animate them using some mathematics with a lot of trigonometry:

```
const clock = new THREE.Clock()

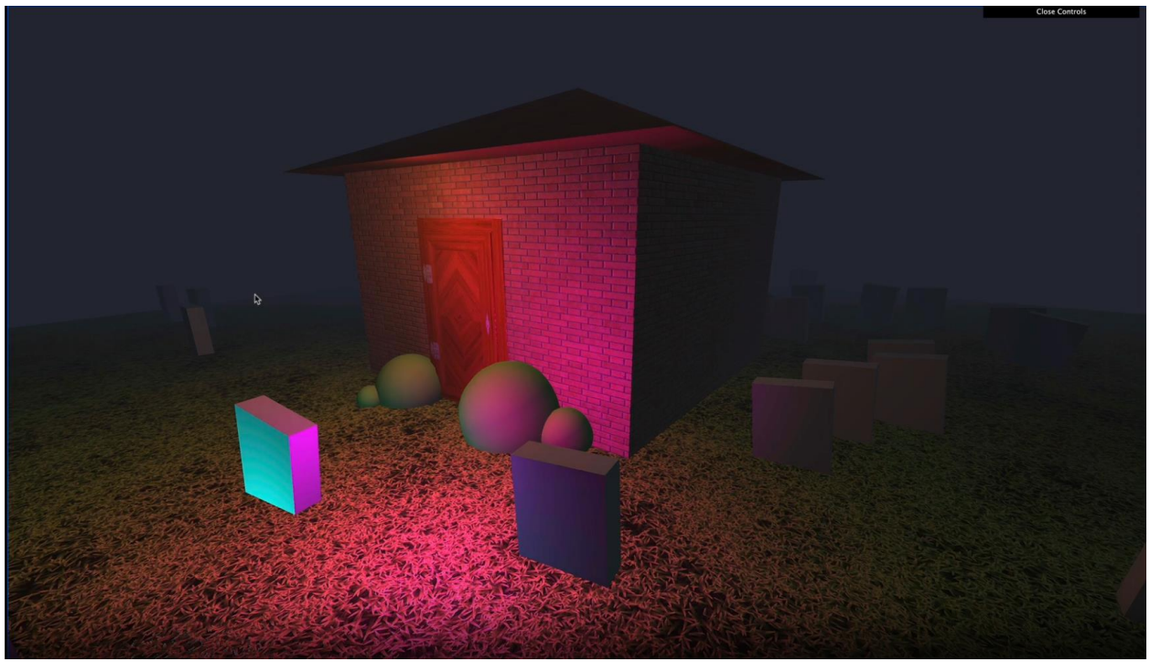
const tick = () =>
{
    const elapsedTime = clock.getElapsedTime()

    // Ghosts
    const ghost1Angle = elapsedTime * 0.5
    ghost1.position.x = Math.cos(ghost1Angle) * 4
    ghost1.position.z = Math.sin(ghost1Angle) * 4
    ghost1.position.y = Math.sin(elapsedTime * 3)

    const ghost2Angle = - elapsedTime * 0.32
    ghost2.position.x = Math.cos(ghost2Angle) * 5
    ghost2.position.z = Math.sin(ghost2Angle) * 5
    ghost2.position.y = Math.sin(elapsedTime * 4) + Math.sin(elapsedTime *
2.5)

    const ghost3Angle = - elapsedTime * 0.18
    ghost3.position.x = Math.cos(ghost3Angle) * (7 + Math.sin(elapsedTime *
0.32))
    ghost3.position.z = Math.sin(ghost3Angle) * (7 + Math.sin(elapsedTime *
0.5))
    ghost3.position.y = Math.sin(elapsedTime * 4) + Math.sin(elapsedTime *
2.5)

    // ...
}
```



Shadows

Finally, to add more realism, let's add shadows.

Activate the shadow map on the renderer:

```
renderer.shadowMap.enabled = true
```

Activate the shadows on the lights that you think should cast shadows:

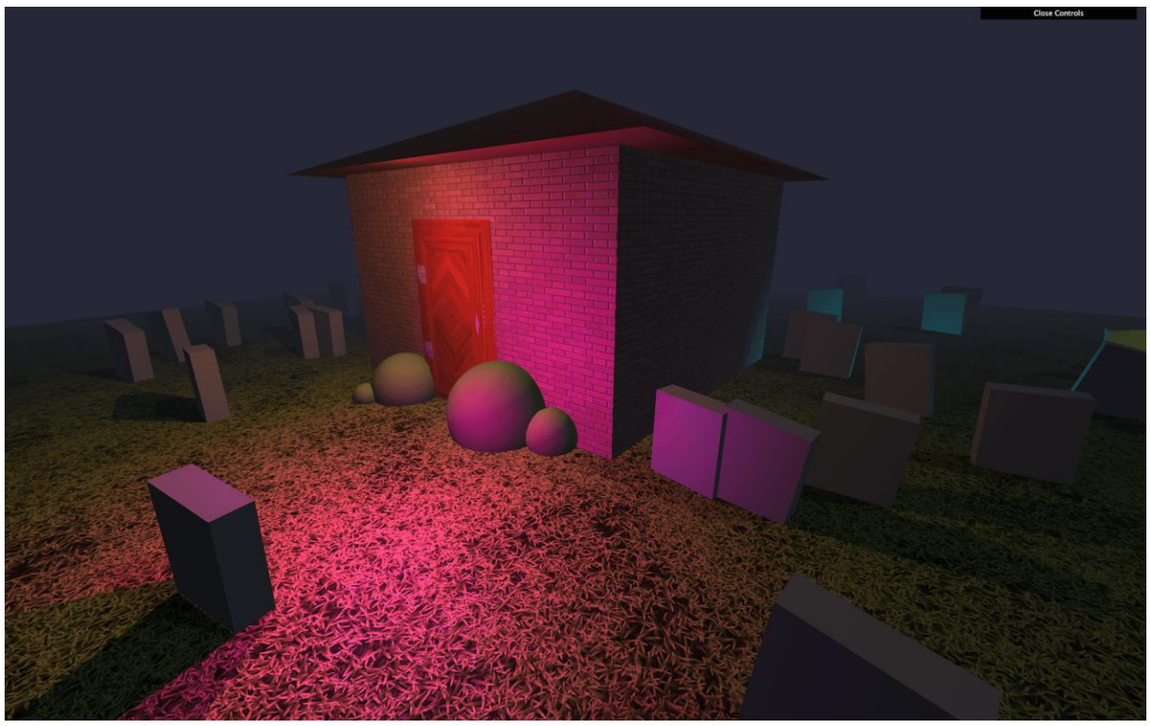
```
moonLight.castShadow = true
doorLight.castShadow = true
ghost1.castShadow = true
ghost2.castShadow = true
ghost3.castShadow = true
```

Go through each object of your scene and decide if that object can cast and/or receive shadows:

```
walls.castShadow = true
bush1.castShadow = true
bush2.castShadow = true
bush3.castShadow = true
bush4.castShadow = true

for(let i = 0; i < 50; i++)
{
    // ...
    grave.castShadow = true
    // ...
}

floor.receiveShadow = true
```



The scene looks much better with these shadows, but we should optimize them anyway.

A good thing would be to go through each light, create camera helpers on the **light.shadowMap.camera**, and make sure the **near**, the **far**, the **amplitude** or the **fov** fit nicely. But instead, let's use the following values that should be just right.

We can also reduce the shadow map render sizes to improve performances:

```
moonLight.shadow.mapSize.width = 256
moonLight.shadow.mapSize.height = 256
moonLight.shadow.camera.far = 15

// ...

doorLight.shadow.mapSize.width = 256
doorLight.shadow.mapSize.height = 256
doorLight.shadow.camera.far = 7

// ...

ghost1.shadow.mapSize.width = 256
ghost1.shadow.mapSize.height = 256
ghost1.shadow.camera.far = 7

// ...

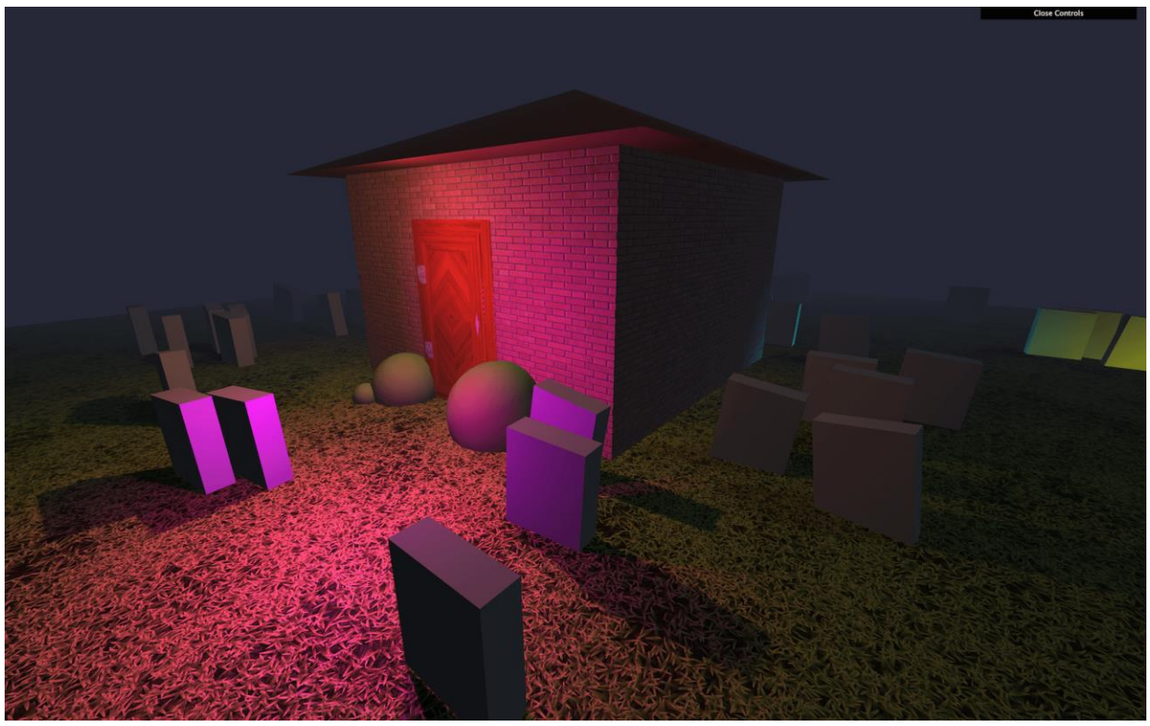
ghost2.shadow.mapSize.width = 256
ghost2.shadow.mapSize.height = 256
ghost2.shadow.camera.far = 7

// ...

ghost3.shadow.mapSize.width = 256
ghost3.shadow.mapSize.height = 256
ghost3.shadow.camera.far = 7

// ...

renderer.shadowMap.type = THREE.PCFSoftShadowMap
```


Close Controls